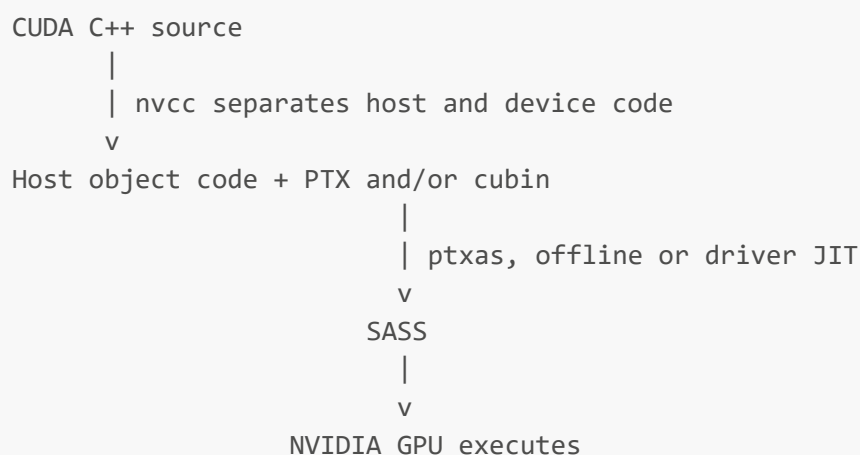


GPU Compiler Stack: From CUDA C++ to Graph Compilers

Advanced GPU work is not only about writing kernels. It is also about understanding how source code becomes machine instructions, when compilation happens, and how frameworks transform whole graphs before any kernel runs.

The central mental model is:



At a higher level, a graph compiler may first rewrite many tensor operations into fewer, better kernels:

```
Model/program -> computation graph -> graph optimizations -> fused kernels  
                                                    -> CUDA/PTX/SASS -> GPU
```

The chapters below cover **PTX, SASS, LLVM basics, CUDA compilation, JIT compilation, kernel fusion, and graph compilers**. Commands and flags are representative; exact availability depends on the installed CUDA toolkit, GPU architecture, driver, and framework version.

LLVM Basics for GPU Compilation

1. Overview

LLVM is a modular compiler infrastructure built around reusable intermediate representations, analyses, optimization passes, and target backends. LLVM is not just one compiler and its name is no longer treated as an acronym. Clang is a C/C++ front end built with LLVM; other languages and systems can also generate LLVM IR and reuse its optimizer and code-generation machinery.

LLVM matters to GPU systems because modern GPU toolchains often need a common representation in which they can analyze control flow, prove memory properties, optimize arithmetic, and lower code toward a device

target. Clang can compile CUDA and HIP-style programs, NVIDIA's toolchain uses NVVM IR related to LLVM IR inside its device pipeline, and graph/tensor compilers frequently use LLVM or MLIR at lower levels.

Real systems use LLVM infrastructure in language compilers, JIT runtimes, shader or kernel compilers, heterogeneous programming stacks, and CPU/GPU code generators. Interviewers ask LLVM basics to test whether a candidate understands compiler stages, SSA form, control-flow graphs, optimization passes, and why a compiler uses several IR levels rather than one representation for everything.

2. Core Idea

Think of LLVM IR as a standardized shipping container between language-specific front ends and hardware-specific backends:

```
C/C++ --Clang--\
Rust -----> LLVM IR -> analyses/optimizations -> target backend -> machine
code
Other language -/                                |-> x86
                                                    |-> ARM
                                                    |-> GPU-related target path
```

A tiny function:

```
int max2(int a, int b) { return a > b ? a : b; }
```

may be expressed conceptually in SSA-like IR as:

```
define i32 @max2(i32 %a, i32 %b) {
entry:
  %cond = icmp sgt i32 %a, %b
  %result = select i1 %cond, i32 %a, i32 %b
  ret i32 %result
}
```

Step by step:

1. A front end parses source and checks language rules.
2. It lowers the source into LLVM IR, with explicit types, operations, blocks, and control flow.
3. Analyses compute facts such as dominance, aliasing, and loop structure.
4. Passes transform IR while preserving observable semantics.
5. A backend selects target instructions, schedules them, and allocates registers.
6. For a GPU path, target-specific lowering must also model address spaces, kernels, thread state, and device calling conventions.

LLVM IR is lower-level than CUDA C++ but still more abstract than final machine code. That middle position makes it reusable.

3. Important Subtopics

3.1 Front end, middle end, and backend

- **Front end:** Parses a language, type-checks it, and emits IR.
- **Middle end:** Runs mostly target-independent analyses and optimizations.
- **Backend:** Lowers to a target instruction set and performs target-dependent optimization.
- **Why it matters:** The separation allows many languages and hardware targets to share work.
- **Example:** Clang handles C++ templates; LLVM optimization handles dead code; a GPU backend handles device instructions.
- **Interview angle:** `nvcc`, Clang, LLVM, PTX generation, and native assembly are stages/components, not synonyms.

3.2 Static Single Assignment form

In **SSA**, each virtual value is assigned once. A new assignment creates a new name.

```
x0 = 1
if cond: x1 = 2
merge: x2 = phi(x0, x1)
```

- **Why it matters:** Def-use relationships are explicit, simplifying constant propagation, dead-code elimination, and many data-flow analyses.
- **Example:** A `phi` node selects a value based on the incoming control-flow edge.
- **Interview angle:** SSA values are not mutable source variables and are not physical registers.

3.3 Basic blocks and control-flow graphs

A basic block is a straight-line instruction sequence with one entry and a terminating control-flow instruction. Edges between blocks form a **CFG**.

- **Why it matters:** Loops, branches, dominance, reachability, and divergence analysis operate on the CFG.
- **Example:** An `if/else` creates condition, then, else, and merge structure.
- **Interview angle:** GPU divergence starts as control flow in the IR but must be understood under SIMT execution.

3.4 Analyses versus transformation passes

An analysis derives facts without changing program meaning; a transformation rewrites the program.

- **Why it matters:** Transformations rely on valid analysis results and may invalidate them.
- **Example:** Alias analysis may prove two pointers independent, enabling load reordering or vectorization.
- **Interview angle:** Pass ordering matters because one transformation can expose or hide opportunities for another.

3.5 Alias analysis and memory effects

Compilers must know whether two pointers may refer to the same storage and whether calls read or write memory.

- **Why it matters:** Uncertain aliasing blocks motion, vectorization, common-subexpression elimination, and fusion-related optimization.
- **Example:** Source-level `restrict`-like promises can give the optimizer stronger independence information when correct.
- **Interview angle:** Incorrect no-alias claims cause undefined behavior or wrong-code opportunities, not merely faster code.

3.6 LLVM IR address spaces

Pointer address spaces distinguish kinds of memory. GPU targets map them to concepts such as global, shared/local-data-share, constant, or thread-private memory according to target conventions.

- **Why it matters:** Address spaces affect legal operations, alias reasoning, and selected instructions.
- **Example:** A shared-memory pointer should lower differently from a generic global pointer.
- **Interview angle:** Numeric address-space identifiers are target-specific conventions; do not memorize one mapping as universal LLVM meaning.

3.7 Intrinsics and metadata

Intrinsics express operations or semantic facts not represented well by ordinary calls. Metadata carries optimization hints or debug/source information.

- **Why it matters:** GPU built-ins and target-specific operations need explicit compiler representation.
- **Example:** Thread identifiers or special math operations can become target intrinsics before lower-level code generation.
- **Interview angle:** Metadata often guides optimization but generally should not change required language semantics.

3.8 LLVM IR, NVVM IR, and MLIR

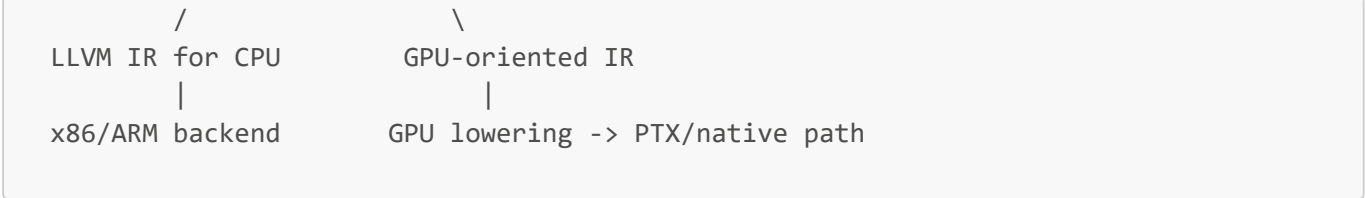
NVVM IR is NVIDIA's LLVM-based device IR with additional conventions and supported features. MLIR is a multi-level IR framework in the LLVM project that can retain tensor, loop, affine, GPU, and target-level concepts through multiple dialects.

- **Why it matters:** LLVM IR is often too low-level for graph-wide tensor transformations, while MLIR can progressively lower high-level structure.
- **Example:** Tensor operations may lower to loop/tile representations, then GPU-specific operations, then LLVM-compatible IR.
- **Interview angle:** LLVM IR and MLIR are complementary layers, not simply competing textual syntaxes.

4. Real-World Example

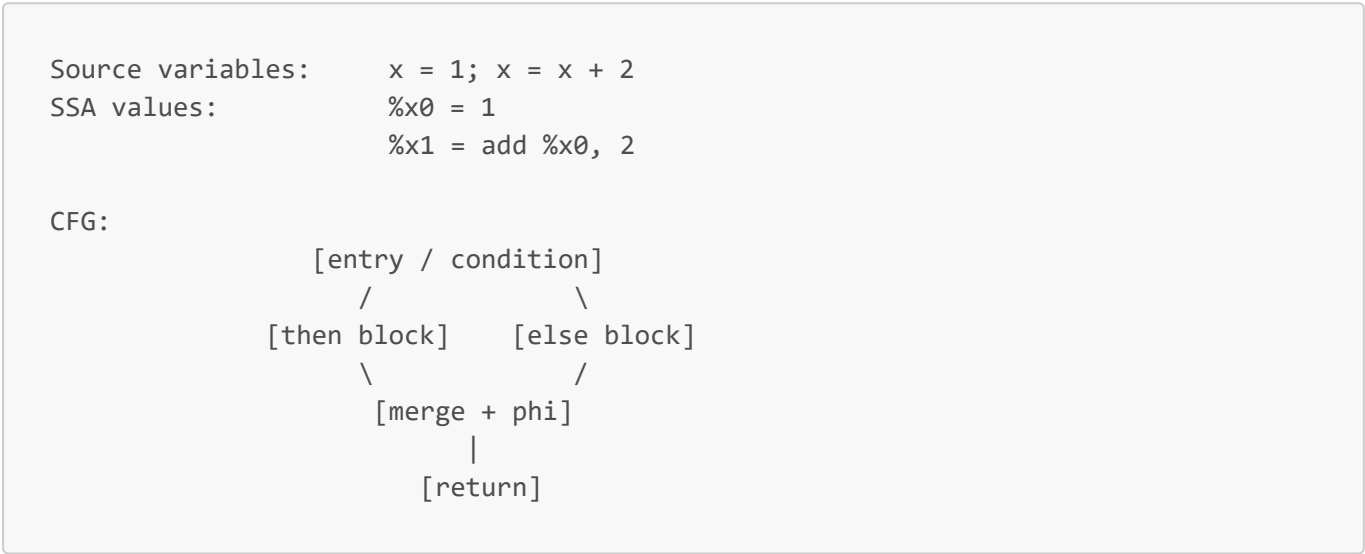
Consider a language runtime that JIT-compile an elementwise expression for both CPU and GPU:

```
User expression: z = relu(a*x + y)
      |
Language AST / typed graph
      |
Target-neutral optimization
```



The front end is shared, but address spaces, thread indexing, vectorization, calling conventions, and code generation differ by target. Keeping target-independent simplification above the split prevents duplicate work; retaining GPU structure until the appropriate lowering stage avoids losing information needed for mapping work to threads and memory.

5. Diagrams / Mental Models



Compiler layer	Preserves	Good transformations
AST/high-level IR	Language and domain intent	Type checking, desugaring, graph rewrites
MLIR/tensor-loop IR	Shapes, loops, tiles, mappings	Fusion, tiling, layout, parallel mapping
LLVM IR	Typed operations, CFG, memory, calls	Scalar simplification, DCE, inlining, lower-level optimization
PTX	Virtual NVIDIA GPU semantics	Target-family lowering
SASS	Exact target instructions	Hardware execution

6. Common Interview Questions

Q1. What is LLVM?

Answer: A modular compiler infrastructure containing IRs, analyses, optimizers, code-generation libraries, and related tools. **Expected:** Distinguish LLVM from a single language compiler. **Common mistake:** Saying LLVM is merely a C++ compiler; Clang is the C/C++ front end.

Q2. Why is an intermediate representation useful?

Answer: It decouples source languages from hardware targets and provides a common form for analysis and optimization. **Expected:** N front ends and M backends avoid $N \times M$ complete compilers. **Common mistake:** Describing IR as only a temporary assembly file.

Q3. What is SSA form?

Answer: A representation where each SSA value has one definition; merges use phi-like selection. **Expected:** Clear def-use chains and optimization value. **Common mistake:** Claiming a source variable can never change.

Q4. What is a basic block?

Answer: A maximal straight-line instruction sequence with control entering at the beginning and leaving through a terminator. **Expected:** Blocks are CFG nodes. **Common mistake:** Calling each source brace-delimited scope a basic block.

Q5. What is a phi node?

Answer: It selects the SSA value associated with the control-flow edge by which execution entered a block. **Expected:** Used at merge points. **Common mistake:** Treating it as an ordinary runtime function call.

Q6. What is the difference between an analysis and an optimization pass?

Answer: Analysis computes facts; a transformation changes IR while preserving semantics. **Expected:** Transformations can invalidate analyses. **Common mistake:** Assuming pass order never matters.

Q7. Why does alias analysis matter for GPU code?

Answer: The compiler needs to know whether memory operations can interfere before reordering, caching, vectorizing, or eliminating them. **Expected:** Connect alias uncertainty to lost optimization. **Common mistake:** Assuming different pointer variable names imply different storage.

Q8. Is LLVM IR portable machine code?

Answer: It is a compiler IR with a data layout and target assumptions, not a universal write-once executable format. **Expected:** It still depends on target triples, layouts, supported features, and lowering rules. **Common mistake:** Treating bitcode as a permanently stable cross-platform application binary.

Q9. How is LLVM relevant to CUDA/GPU compilation?

Answer: LLVM-based front ends and infrastructures can represent and optimize device code; NVIDIA's NVVM IR is LLVM-related, and GPU compilers often lower through LLVM/MLIR layers before PTX or native code. **Expected:** Avoid claiming every `nvcc` stage is open-source LLVM. **Common mistake:** Equating LLVM IR directly with PTX.

Q10. Why might a graph compiler use MLIR before LLVM IR?

Answer: MLIR dialects can retain shapes, tensors, affine loops, layouts, and GPU mappings that ordinary low-level LLVM IR would obscure. **Expected:** Progressive lowering. **Common mistake:** Lowering early and expecting low-level passes to rediscover all domain structure.

7. Deep-Dive Questions

1. Why does pass ordering matter?

Inlining may expose constants; constant propagation may simplify branches; loop canonicalization may expose vectorization. Reversing those passes can miss opportunities. Pass pipelines therefore balance enabling transformations, compile time, and code-size growth.

2. How does dominance relate to SSA?

A definition must dominate its ordinary uses: every path to the use passes through the definition. Phi operands are associated with predecessor edges, which handles values defined along different paths.

3. Why is GPU address-space lowering difficult?

Pointers with different visibility and storage rules may require distinct instructions. Generic pointers, casts, alias relationships, and target-specific mappings must remain correct, and careless address-space conversion can lose optimization facts or change semantics.

4. Why not perform all GPU optimization on LLVM IR?

After tensors become pointer arithmetic and loops, high-level information such as broadcast dimensions, layout choices, and operator boundaries is harder to recover. Fusion, tiling, and tensor-layout reasoning are usually easier in richer IRs; LLVM IR remains strong for lower-level work.

5. How does undefined behavior affect optimization?

The optimizer may assume a valid program does not execute undefined behavior. It can remove or reorder code based on that assumption. Invalid aliasing promises, out-of-bounds access, or signed overflow assumptions can therefore yield surprising but legal transformations.

8. Comparison Tables

Aspect	Clang	LLVM middle end	LLVM backend
Main job	Parse/check language and emit IR	Analyze and optimize IR	Select target instructions and allocate resources
Knows deeply about	C/C++ syntax and semantics	CFG, values, memory properties	Target ISA and machine model
Typical output	LLVM IR	Optimized LLVM IR	Object/machine code or target representation

Aspect	LLVM IR	MLIR	PTX
Abstraction	Low-level typed SSA	Multiple extensible abstraction levels	NVIDIA virtual GPU ISA
Domain structure	Limited after lowering	Can preserve tensors, affine loops, GPU ops	Kernel/thread/memory instructions
Main role	Reusable optimizer/backend input	Progressive multi-level lowering	NVIDIA target boundary and JIT input

Aspect	LLVM IR	MLIR	PTX
Hardware scope	Many targets through backends	Many domains/targets through dialect conversion	NVIDIA GPUs

9. Common Mistakes

- Using LLVM and Clang as interchangeable terms.
- Describing SSA values as physical registers.
- Assuming LLVM bitcode is a stable universal executable format.
- Ignoring target data layout and address spaces.
- Expecting low-level IR to retain all tensor/graph intent.
- Assuming optimizations are safe when source code violates its language contract.
- Claiming `nvcc` is simply Clang or that PTX is LLVM IR.

10. Edge Cases / Special Cases

- Memory form may temporarily contain loads/stores before `mem2reg`-style promotion creates cleaner SSA values.
- Irreducible control flow, exceptions, indirect calls, and inline assembly complicate analysis.
- Volatile and atomic operations restrict otherwise legal reordering and elimination.
- GPU backends have target-specific intrinsics, metadata, calling conventions, and address-space rules.
- Optimization can increase code size or register pressure even when it reduces source-level operation count.
- Fast-math flags change the algebraic transformations a compiler may legally perform.

11. How to Explain in Interview

LLVM is modular compiler infrastructure organized around typed SSA-based IR, analyses, optimization passes, and target backends. A front end such as Clang handles source-language semantics; LLVM's middle end performs reusable optimization; a backend lowers to a target. In GPU stacks, LLVM-related and MLIR representations are useful before lowering to PTX or native code because they separate high-level optimization from target-specific code generation.

12. Quick Revision Notes

- Clang is a front end; LLVM is broader infrastructure.
- SSA gives each IR value one definition; phi nodes merge control-flow values.
- Basic blocks form a CFG.
- Analyses provide facts; transformations rewrite IR and can invalidate facts.
- Alias and memory-effect information determine which memory optimizations are legal.
- MLIR retains higher-level domain structure; LLVM IR is a common lower-level boundary.
- Interview trap: IR portability is constrained by data layout, targets, and compatibility.

13. Practice Tasks

1. Use Clang to emit LLVM IR for a small C++ function at `-O0` and `-O2`; compare loads, stores, and SSA values.

2. Draw the CFG and phi placement for a loop containing an `if/else`.
3. Remove `restrict`-like information from a vector loop and inspect how optimization changes.
4. Use LLVM's optimizer tooling, where installed, to run one named pass and inspect the result.
5. Explain which transformations belong at graph, loop/tensor, LLVM, PTX, and SASS levels.
6. Identify the target-specific assumptions in a piece of LLVM IR: triple, data layout, intrinsics, and address spaces.

14. Final Cheat Sheet

Item	Remember
Core definition	Modular IR, optimization, and code-generation infrastructure
Why it matters	Reuses compiler work across languages and targets
Most asked	SSA, phi nodes, CFG, passes, alias analysis
Key comparison	MLIR preserves higher-level structure; LLVM IR supports lower-level reusable optimization
Biggest trap	LLVM is not synonymous with Clang or with one executable compiler
One-line answer	"LLVM provides SSA-based IR and reusable compiler passes/backends that GPU toolchains can use before target-specific lowering."

CUDA Compilation

1. Overview

CUDA compilation is the multi-stage process that turns a mixed CUDA C++ translation unit into host code for the CPU and device code for NVIDIA GPUs, then packages and links those parts so the host can register and launch kernels.

`nvcc` is a **compiler driver**: it coordinates preprocessing, CUDA-specific source separation/transformation, a supported host compiler, device compilation, assembly, device linking when needed, and packaging. The exact internal pipeline is toolchain-dependent, but the stable interview model is host/device separation plus target-specific device-code generation.

It matters because build flags decide GPU compatibility, binary size, startup behavior, optimization, debug visibility, and whether calls across translation units can link. Real systems use fat binaries to support fleets with multiple GPU generations. Interviewers ask about CUDA compilation to test whether candidates can reason across language compilation, linking, runtime loading, and GPU architecture targets.

2. Core Idea

A `.cu` file contains two worlds:

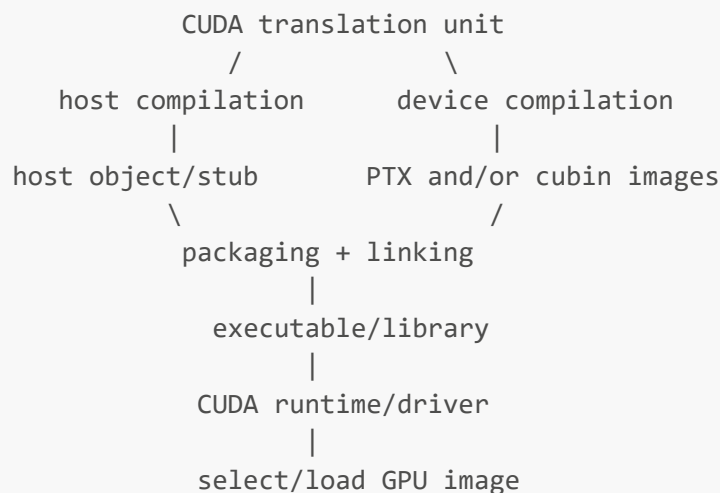
```

__global__ void add(const float* a, const float* b, float* c) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    c[i] = a[i] + b[i];
}

int main() {
    // CPU code configures data and launches GPU work.
    add<<<grid, block>>>(a, b, c);
}

```

Conceptually:



Step by step:

1. Preprocessing and CUDA parsing identify host functions, device functions, kernels, and launch syntax.
2. Host-side launch code is transformed into runtime calls and registration stubs.
3. A host compiler compiles CPU code.
4. Device code is optimized and compiled for requested virtual and/or real architectures.
5. Cross-translation-unit device references are resolved through device linking when relocatable device code is used.
6. Device images are embedded in the host object/library.
7. At runtime, the loader selects a compatible cubin or JIT-compiles embedded PTX.

3. Important Subtopics

3.1 Execution-space qualifiers

`__host__`, `__device__`, and `__global__` describe where functions compile and how they are called.

- `__global__`: Kernel entry, launched from host (and in supported cases from device mechanisms), returns `void` under ordinary CUDA kernel rules.
- `__device__`: Runs on GPU and is called by device code.
- `__host__`: Runs on CPU; default for ordinary functions.

- **Why it matters:** Host and device environments have different APIs, instructions, and compilation constraints.
- **Interview angle:** A `__host__ __device__` function is compiled into separate host and device versions; it is not one universal binary body.

3.2 `nvcc` as a driver

`nvcc` coordinates multiple tools rather than replacing every phase with one monolithic compiler.

- **Why it matters:** Host compiler compatibility and options influence the CPU half; CUDA options influence device targets and packaging.
- **Example:** `-Xcompiler` passes an option to the host compiler; `-Xptxas` passes options to the device assembler.
- **Interview angle:** Know which stage owns an error or optimization decision.

3.3 Virtual and real architecture code generation

Code-generation specifications choose whether to embed PTX, cubin code, or both for particular target levels.

```
virtual target: compute_xy -> PTX feature contract
real target:    sm_xy      -> native cubin for that SM target
```

- **Why it matters:** It controls compatibility, startup latency, and package size.
- **Interview angle:** A production build often includes native code for common GPUs and PTX as a fallback when appropriate.

3.4 Fat binaries and runtime selection

A fat binary stores multiple device images inside one host artifact.

- **Why it matters:** One application can support heterogeneous deployment fleets.
- **Example:** Native `sm_80` and `sm_90` code plus a selected PTX fallback.
- **Interview angle:** More images increase binary size and build time.

3.5 Separate compilation and device linking

Without relocatable device code, the compiler generally needs device-call visibility appropriate for whole-translation-unit compilation/inlining. Separate compilation preserves device references for a later device-link step.

- **Why it matters:** Large C++ projects place device functions in multiple translation units.
- **Example:** Compile with relocatable device code, then perform a device link before the final host link.
- **Interview angle:** Host linking alone cannot resolve the GPU-side call graph encoded in device objects.

3.6 Compilation outputs

- PTX: virtual device code.
- Cubin: native device binary for target architecture.
- Object file: host object plus embedded/registered device data.

- Fatbin: packaged collection of device images.
- **Why it matters:** Each artifact answers a different debugging/deployment need.
- **Interview angle:** `-ptx` is useful for inspection but does not build a normal complete host executable.

3.7 Optimization, debug, and line information

Host and device compilation have separate debug/optimization concerns.

- **Why it matters:** Full debug modes can change code generation; profiler-friendly line mapping need not require fully unoptimized device code.
- **Example:** Resource reports from `ptxas` expose registers, stack, constant memory, and spills.
- **Interview angle:** Always compare performance builds with equivalent optimization settings.

3.8 Static CUDA runtime versus driver API loading

The CUDA Runtime API offers language-integrated launch and registration conveniences. The Driver API explicitly manages contexts, modules, functions, and launches.

- **Why it matters:** JIT systems and plugin-like loaders often need explicit module loading.
- **Example:** A runtime may compile PTX and use the Driver API to load a module and retrieve a kernel function.
- **Interview angle:** Both ultimately rely on the driver; they differ in abstraction and lifecycle control.

4. Real-World Example

A deployment must run on three known GPU classes and remain usable on a later compatible class. The build embeds native cubins for the three known targets and PTX for a deliberate virtual target. On startup:

```
Detect device
|
+-- matching native cubin? --> load immediately
|
+-- otherwise compatible PTX? --> driver JIT --> cache --> launch
|
+-- neither? --> report no compatible kernel image
```

This is preferable to shipping PTX alone when cold-start latency matters, and preferable to native code alone when future deployment hardware is not fully known.

5. Diagrams / Mental Models

Compile-time choices	Runtime result
sm_80 cubin ----->	exact compatible target loads native code
sm_90 cubin ----->	exact compatible target loads native code
compute_90 PTX ----->	compatible driver compiles for actual GPU
no compatible image ----->	module/load or launch failure

Stage	Input	Output	Typical failure
Host compile	Transformed host C++	Host object	Unsupported host compiler or C++ error
Device front/middle end	Device C++	Device IR/PTX-like form	Unsupported device construct/type error
Device assembly	PTX/device IR	Cubin/native code	Target mismatch or resource/codegen error
Device link	Relocatable device objects	Linked device image	Unresolved device symbol
Host link/package	Host objects + device data	App/library	Host symbol or library error
Runtime load	Embedded image + current device	Loaded module	No compatible image/PTX JIT incompatibility

6. Common Interview Questions

Q1. What exactly is `nvcc`?

Answer: A CUDA compiler driver that orchestrates CUDA parsing/transformation, host compilation, device compilation, assembly, linking, and packaging. **Expected:** Multi-stage driver. **Common mistake:** Describing it as only a PTX-to-machine-code assembler.

Q2. How does one `.cu` file produce CPU and GPU code?

Answer: CUDA compilation separates host and device portions. Host launch syntax becomes runtime-facing code/stubs, while kernels and device functions enter the GPU compilation path. The results are packaged together. **Expected:** Two compilation trajectories. **Common mistake:** Saying the CPU executes the kernel body.

Q3. What do `__host__`, `__device__`, and `__global__` mean?

Answer: They select CPU compilation, GPU device-function compilation, and GPU kernel-entry compilation respectively; combined qualifiers can produce multiple versions. **Expected:** Callability and execution space. **Common mistake:** Treating `__global__` as just another device helper.

Q4. What is a fat binary?

Answer: A package containing multiple GPU code images—commonly cubins for several SM targets and optionally PTX—embedded in or associated with a host artifact. **Expected:** Runtime selection. **Common mistake:** Calling every CUDA executable a single-architecture binary.

Q5. Why embed both cubin and PTX?

Answer: Cubin provides known native code and avoids JIT for supported GPUs; PTX can provide a compatibility path for later supported GPUs. **Expected:** Trade startup/size/coverage. **Common mistake:** Claiming PTX fallback is unconditional across all driver/toolkit combinations.

Q6. What is relocatable device code?

Answer: Device code compiled so GPU-side symbols can remain relocatable and be resolved by a later device-link step. **Expected:** Needed for cross-translation-unit device references in relevant builds. **Common mistake:** Assuming the ordinary host linker resolves device calls.

Q7. What is device linking?

Answer: It combines device-code objects, resolves GPU symbols, and produces a linked device image before or during final packaging. **Expected:** Separate from host linking. **Common mistake:** Confusing it with copying data to a device.

Q8. When is PTX JIT performed?

Answer: At module load or first use when the runtime/driver selects PTX rather than a compatible embedded native image. **Expected:** Driver generates native code and may cache it. **Common mistake:** Saying every kernel is JIT-compiled on every launch.

Q9. How do you check resource usage during compilation?

Answer: Request verbose device-assembler/resource information and inspect registers, stack, spill loads/stores, shared/constant memory, then validate runtime impact with profiling. **Expected:** Static report plus dynamic validation. **Common mistake:** Treating register count as a complete performance score.

Q10. Why can "no kernel image is available" occur?

Answer: The packaged native images do not match the current GPU and there is no compatible PTX image the driver can compile. **Expected:** Connect build targets to deployed hardware. **Common mistake:** Diagnosing it purely as an invalid launch configuration.

7. Deep-Dive Questions

1. What does `__host__` `__device__` imply for templates and called functions?

The compiler instantiates/checks code for both environments as applicable. A body valid on the host may call facilities unavailable on the device or vice versa, causing one compilation path to fail. Conditional compilation may be needed, but duplicated behavior should remain semantically consistent.

2. Why can separate device compilation reduce optimization opportunity?

Without cross-module visibility, early compilation may not inline or specialize across translation units. Device link-time optimization can recover some opportunities, at added compile/link complexity and time.

3. How does the loader choose between multiple images?

It selects code compatible with the active device, preferring usable native images according to loader compatibility rules; otherwise it may select PTX and invoke JIT. The application should not assume the first embedded image wins.

4. What makes a CUDA build reproducible?

Pin the toolkit, host compiler, flags, target list, libraries, and build environment; prefer embedded native code when exact offline code generation matters. Driver JIT can vary with deployed driver/backend versions even from the same PTX.

5. How do template instantiation and device linking interact?

Templates need definitions visible where instantiated unless explicit instantiation is arranged. Device linking resolves emitted device symbols, but it cannot instantiate a template whose definition was never available to the compiling translation unit.

8. Comparison Tables

Aspect	Whole-program device compilation	Relocatable device code
Device calls	Usually resolved within visible unit/path	May cross translation units
Device-link step	Minimal/not separately required in simple build	Required
Optimization visibility	Strong within compilation unit	May need device LTO for cross-unit optimization
Build complexity	Lower	Higher
Best for	Small/self-contained kernels	Modular device-code projects

Artifact	Contains	Portable across GPU targets?	Executed directly?
PTX	Virtual instructions	Within compatibility constraints	No, normally JIT/assembled first
Cubin	Native device image	Architecture-specific	Yes
Fatbin	Multiple PTX/cubin images	Covers packaged target set	Loader chooses contained image
Host object	CPU code plus device registration/data	Host-platform-specific	Host CPU executes host portion

9. Common Mistakes

- Treating `nvcc` as one opaque source-to-SASS pass.
- Confusing virtual `compute_xy` and real `sm_xy` targets.
- Shipping only one cubin and expecting universal GPU compatibility.
- Assuming PTX guarantees compatibility with arbitrarily old drivers.
- Forgetting device linking for cross-translation-unit device calls.
- Using debug flags while interpreting production resource/performance behavior.
- Forgetting that host and device code can use different compilers and constraints.
- Assuming JIT occurs on every kernel launch.

10. Edge Cases / Special Cases

- Header-only `__host__ __device__` utilities may compile twice and encounter different overloads or macros.
- Static libraries containing relocatable device code may require special care so device objects participate in device linking.
- Runtime compilation APIs compile device source or intermediate code separately from a normal `nvcc` host build.
- Targeting only very new PTX can exclude machines whose drivers cannot parse that PTX version.
- Link-time optimization, dynamic parallelism, debug mode, and whole-program assumptions alter the pipeline.
- C++ ABI compatibility chiefly affects host objects/libraries; device ABI/linking has its own toolchain contracts.

11. How to Explain in Interview

CUDA compilation splits a mixed `.cu` translation unit into host and device paths. `nvcc` coordinates a host compiler for CPU code and NVIDIA device compilation for kernels, then packages PTX and/or architecture-specific cubins with host launch stubs. At runtime the driver loads a compatible cubin or JIT-compiled PTX. For device calls across translation units, relocatable device code and a device-link step may be required.

12. Quick Revision Notes

- `nvcc` is a compiler driver.
- CUDA source has separate host and device compilation paths.
- PTX targets `compute_xy`; cubin/native code targets `sm_xy`.
- Fat binaries cover multiple architectures.
- Native image: fast startup and pinned codegen; PTX: JIT/future compatibility path.
- Device linking resolves device-side symbols across objects.
- Interview trap: host linking and device linking are not the same operation.

13. Practice Tasks

1. Build one kernel as PTX, cubin, object, and executable; list what each artifact contains.
2. Use `--keep` or equivalent intermediate preservation and trace host/device outputs.
3. Create two `.cu` files with one cross-file device function; build with and without relocatable device code and explain the result.
4. Embed two real-architecture images plus PTX; inspect them with binary utilities.
5. Compare resource reports for two block sizes or unroll factors.
6. Run on a supported GPU, determine whether a native image or PTX JIT path was selected using appropriate loader/JIT diagnostics.

14. Final Cheat Sheet

Item	Remember
Core definition	Split host/device compilation coordinated by <code>nvcc</code>
Why it matters	Determines correctness, target coverage, startup, and code quality

Item	Remember
Most asked	<code>nvcc</code> role; fatbins; <code>compute</code> vs <code>sm</code> ; device linking
Key comparison	Native cubin versus embedded PTX
Biggest trap	A <code>.cu</code> file does not become one homogeneous CPU/GPU object stream
One-line answer	" <code>nvcc</code> coordinates separate host and GPU compilation, packages PTX/cubins, and lets the runtime load code for the active GPU."

PTX

1. Overview

PTX (Parallel Thread Execution) is NVIDIA's virtual instruction set and intermediate representation for GPU programs. It is assembly-like text, but it is not usually the final instruction stream executed by an NVIDIA GPU. PTX describes operations, registers, address spaces, thread identifiers, and control flow for a virtual GPU. NVIDIA's assembler or driver later lowers it to architecture-specific machine code.

PTX matters because it is the portability boundary between CUDA source and different generations of NVIDIA GPUs. A library can ship PTX and let a newer driver generate machine code for a compatible future GPU. Compiler engineers inspect PTX to understand address-space conversions, memory instructions, arithmetic choices, register types, and whether source-level transformations survived lowering.

Real systems use PTX in CUDA libraries, JIT-generated kernels, language runtimes such as Numba, framework compilers, and tools that generate GPU code without using CUDA C++ as their source language. Interviewers ask about PTX to check whether a candidate understands intermediate representations, virtual versus physical ISAs, forward compatibility, and the difference between code generation and execution.

2. Core Idea

Think of PTX as a portable recipe and SASS as the exact button presses for one model of appliance. The recipe says "multiply these values and store the result"; the final machine instructions depend on the GPU generation that will perform it.

For a CUDA kernel:

```
__global__ void saxpy(float a, const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = a * x[i] + y[i];
}
```

the important PTX ideas look approximately like this:

```

.visible .entry saxpy(...) {
    // Read virtual special registers representing launch coordinates.
    mov.u32 %r1, %ctaid.x;
    mov.u32 %r2, %ntid.x;
    mov.u32 %r3, %tid.x;
    mad.lo.s32 %r4, %r1, %r2, %r3;

    // Bounds check, load x[i] and y[i], compute, then store y[i].
    setp.ge.s32 %p1, %r4, n;
    @%p1 bra DONE;
    ...
DONE:
    ret;
}

```

Step by step:

1. CUDA C++ expresses a kernel in a high-level language.
2. The CUDA front end resolves C++ semantics and GPU built-ins.
3. Optimization produces a device intermediate form.
4. The backend emits PTX for a chosen **virtual architecture**, such as `compute_80`.
5. `ptxas` or the CUDA driver translates PTX into machine code for a **real architecture**, such as `sm_80`.
6. The GPU executes that machine code, not the textual PTX.

PTX uses an unlimited-looking set of virtual registers such as `%r1` and `%f2`. The final compiler maps live values to finite hardware registers and may spill excess values to local memory.

3. Important Subtopics

3.1 Virtual ISA and versioning

PTX targets a virtual compute capability. A `.version` directive identifies the PTX ISA version, while `.target` identifies required target features.

- **Why it matters:** A driver must understand the PTX version before it can JIT-compile the module.
- **Example:** PTX emitted by a very new toolkit may not be understood by an old installed driver.
- **Interview angle:** Forward compatibility does not mean every driver accepts every future PTX version; the driver still needs a compatible PTX front end.

3.2 Typed virtual registers and predicates

PTX declares registers by type class, such as `.b32`, `.u64`, `.f32`, and `.pred`. Predicates hold boolean conditions and guard instructions.

```
.reg .f32 %f<4>;
.reg .pred %p<2>;
setp.lt.s32 %p1, %r1, %r2;
@%p1 add.f32 %f3, %f1, %f2;
```

- **Why it matters:** Types control instruction semantics, but a register's physical allocation happens later.
- **Interview angle:** PTX register count is useful but is not identical to final hardware register usage.

3.3 Address spaces

PTX distinguishes `.global`, `.shared`, `.local`, `.const`, and `.param` memory spaces. Instructions such as `ld.global`, `ld.shared`, and `st.global` state the space being accessed.

- **Why it matters:** GPU memory spaces differ greatly in visibility, lifetime, latency, and capacity.
- **Example:** Block-cooperative data normally uses `.shared`; thread-private spills appear in `.local` even though local memory is physically backed by device memory/cache.
- **Interview angle:** "Local" means per-thread addressability, not necessarily an on-chip location.

3.4 Special registers

Read-only special registers expose execution state: `%tid`, `%ctaid`, `%ntid`, `%nctaid`, lane ID, clock values, and others.

- **Why it matters:** They connect the abstract kernel to the launch grid and physical execution context.
- **Example:** `mov.u32 %r1, %tid.x` reads the thread's x-coordinate within its block.
- **Interview angle:** Grid indexing is compiled into reads of these special registers plus integer arithmetic.

3.5 Predication and control flow

An instruction may be conditionally executed with `@predicate`, while branches use targets and predicate conditions.

- **Why it matters:** Short conditions may become predicated instructions; larger divergent regions usually require branches and reconvergence machinery.
- **Example:** `@%p1 st.global.u32 [addr], %r2;` stores only when `%p1` is true.
- **Interview angle:** Predication can avoid a branch, but inactive lanes still consume issue opportunity for predicated-off instructions.

3.6 Memory ordering and synchronization

PTX contains barriers, fences, atomic operations, scopes, and memory-order semantics.

- **Why it matters:** Correct parallel code needs both execution synchronization and memory visibility guarantees.
- **Example:** A block barrier coordinates threads in one CTA; it is not a grid-wide barrier for an ordinary kernel.
- **Interview angle:** A barrier and a memory fence solve related but different problems.

3.7 Inline PTX

CUDA C++ supports inline PTX through `asm`, mainly for operations not conveniently exposed by the language or intrinsics.

```
__device__ unsigned lane_id() {
    unsigned id;
    asm("mov.u32 %0, %%laneid;" : "=r"(id));
    return id;
}
```

- **Why it matters:** It provides low-level control but couples code to PTX constraints and toolchain behavior.
- **Interview angle:** Prefer CUDA intrinsics when available; inline PTX can inhibit portability and optimizations.

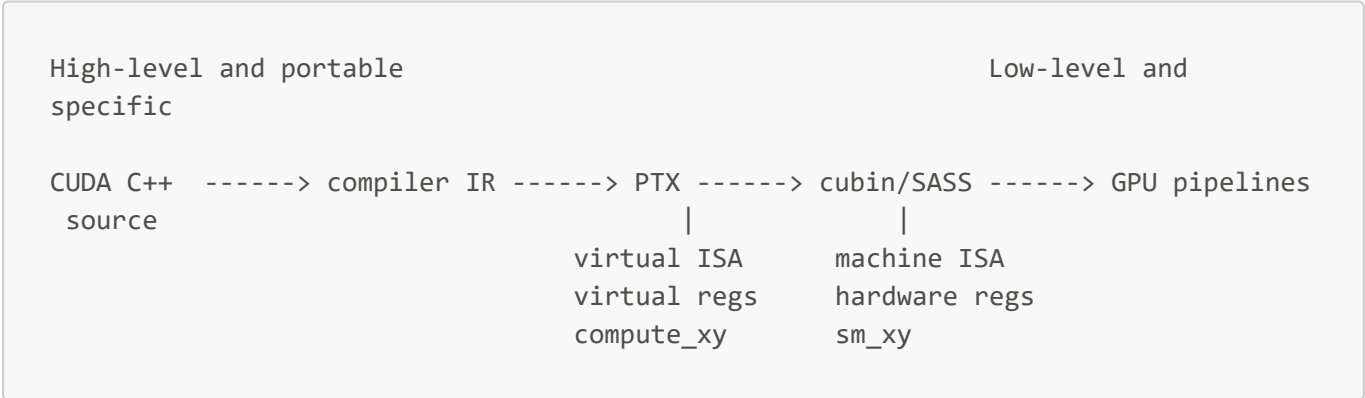
4. Real-World Example

Suppose an inference service ships a CUDA kernel library to customers with several GPU generations. It can package:

```
sm_80 cubin -> starts immediately on Ampere-class target
sm_90 cubin -> starts immediately on Hopper-class target
compute_90 PTX -> driver can JIT for a later compatible architecture
```

The native cubins give predictable startup and known code generation on explicitly supported devices. The PTX entry provides a forward-compatibility path. On first use, the driver translates PTX to SASS and may cache the result. This design trades package size against architecture coverage and JIT latency.

5. Diagrams / Mental Models



PTX element	Mental model	Typical clue
<code>.entry</code>	Kernel entry point	Host can launch it
<code>.func</code>	Device function	Called from device code
<code>%tid.x</code>	Thread coordinate	Part of global index calculation

PTX element	Mental model	Typical clue
<code>.pred</code>	Boolean guard	Predication or branch condition
<code>ld.global</code>	Device-memory load	Check access width and pattern
<code>ld.shared</code>	Shared-memory load	Check bank behavior and synchronization
<code>.local</code>	Per-thread address space	May indicate stack data or spills

6. Common Interview Questions

Q1. Is PTX the machine code executed by an NVIDIA GPU?

Answer: Usually no. PTX is a virtual ISA. `ptxas` or the driver backend turns it into architecture-specific SASS stored in a cubin or generated at JIT time. **Expected:** Distinguish virtual ISA from native ISA. **Common mistake:** Calling PTX "GPU assembly" without qualifying that it is virtual assembly.

Q2. Why does NVIDIA use PTX instead of compiling CUDA C++ directly to every GPU?

Answer: PTX creates a stable compiler boundary. Language front ends can target one virtual representation, while NVIDIA's backend maps it onto particular GPU generations. It also supports driver JIT and forward compatibility. **Expected:** Portability, delayed optimization, multiple front ends. **Common mistake:** Claiming the same PTX always produces identical machine code on every GPU.

Q3. What is the difference between `compute_80` and `sm_80`?

Answer: `compute_80` denotes a virtual architecture used when producing PTX; `sm_80` denotes a real GPU architecture used when producing native cubin code. **Expected:** Virtual target versus binary target. **Common mistake:** Treating both flags as aliases.

Q4. Are PTX registers physical GPU registers?

Answer: No. They are typed virtual registers. Backend register allocation maps live virtual values to a finite physical register file and may spill. **Expected:** Mention liveness and allocation. **Common mistake:** Predicting occupancy from PTX register declarations alone.

Q5. What does PTX local memory mean?

Answer: It is a thread-private address space used for items such as arrays, stack frames, or spills. It is normally backed by device memory and served through caches, not guaranteed on-chip like a register. **Expected:** Addressability versus physical location. **Common mistake:** Equating "local" with low latency.

Q6. What is predication?

Answer: A predicate register guards whether an instruction has an effect for a lane. It can replace short branches, although predicated-off lanes do not perform the operation and the warp still issues the instruction. **Expected:** SIMT lane behavior. **Common mistake:** Saying predication makes divergent work free.

Q7. Can PTX provide forward compatibility?

Answer: Yes, if the application embeds suitable PTX and the installed driver understands its PTX version and can target the new GPU. **Expected:** State the driver-version constraint. **Common mistake:** Assuming PTX generated by any future toolkit works on any old driver.

Q8. Why inspect PTX?

Answer: To verify instruction classes, memory address spaces, conversions, vector widths, control flow, atomics, and compiler transformations. Final performance diagnosis still needs SASS and profiling. **Expected:** PTX is one diagnostic layer. **Common mistake:** Treating PTX inspection alone as a performance measurement.

Q9. When should inline PTX be used?

Answer: When a needed capability is not exposed reliably through CUDA C++ or intrinsics, and the portability/maintenance cost is acceptable. **Expected:** Prefer supported intrinsics first. **Common mistake:** Using inline PTX for ordinary arithmetic in the hope that it always beats the compiler.

Q10. How do barriers and fences differ in PTX-level reasoning?

Answer: A barrier coordinates participating threads at an execution point and commonly includes specified memory-order effects; a fence orders or makes a thread's memory operations visible at a scope but does not make other threads wait. **Expected:** Execution rendezvous versus memory ordering. **Common mistake:** Using a fence as if it were a block barrier.

7. Deep-Dive Questions

1. Why can optimized PTX still differ greatly from SASS?

The backend performs target-dependent instruction selection, scheduling, register allocation, dependency handling, and sometimes additional optimization. One PTX operation may expand into several SASS instructions or combine with adjacent work.

2. How can PTX-level register pressure mislead you?

Virtual register declarations do not show the final interference graph. Values with non-overlapping lifetimes can share physical registers, while backend expansion can introduce temporaries. Use the assembler's register report and profiler data for the final answer.

3. What happens when PTX uses a feature unsupported by the target GPU?

Compilation can fail, or the toolchain may lower an operation to a supported sequence when the ISA contract permits it. Selecting a virtual target promises that required features are compatible with that target level; it is not permission to run arbitrary new operations on old hardware.

4. Why is memory-space information valuable in an IR?

It exposes aliasing and storage semantics to optimization and selects different hardware paths. A compiler can reason differently about block-private shared memory, immutable constant data, and globally visible device memory.

5. Can hand-written PTX guarantee exact instructions?

No. PTX is still input to a backend. If exact native instructions and scheduling matter, SASS must be inspected, and even then runtime performance depends on data, occupancy, memory behavior, and hardware scheduling.

8. Comparison Tables

Aspect	PTX	SASS	
Level	Virtual ISA	Native GPU ISA	
Target	<code>compute_xy</code>	<code>sm_xy</code>	
Representation	Usually readable text	Binary, shown through disassembly	
Registers	Virtual, typed	Physical register references	
Portability	Can be JIT-compiled for compatible later GPUs	Specific to an architecture family/target	
Generated by	CUDA/device backend front portion	<code>ptxas</code> or driver backend	
Executed directly	Normally no	Yes	
Best use in analysis	Semantics and mid-level lowering	Actual instruction selection and scheduling	
Address space	Visibility	Typical use	Important trap
Global	Grid/device-visible through pointers	Large arrays	Coalescing and cache behavior matter
Shared	Threads in a block/CTA	Tiling, reuse	Requires correct synchronization
Local	One thread	Stack objects, spills	Physically not necessarily on-chip
Constant	Read-only kernel-visible data	Uniform parameters/tables	Fastest when warp accesses are favorable
Parameter	Kernel/function arguments	Launch parameters	Often copied/loaded through a parameter path

9. Common Mistakes

- Calling PTX the final hardware assembly without qualification.
- Assuming each PTX instruction maps one-to-one to SASS.
- Treating virtual-register count as the final register count.
- Believing `.local` data is always on-chip.
- Mixing up PTX version, virtual compute capability, and physical SM architecture.
- Using inline PTX when an intrinsic is clearer and equally capable.
- Ignoring driver compatibility when relying on PTX JIT.
- Optimizing based only on PTX instead of confirming with SASS and profiling.

10. Edge Cases / Special Cases

- A fat binary may contain several cubins plus PTX; the loader chooses the best compatible image.
- Debug, line-info, optimization level, relocatable device code, and link-time optimization can change emitted PTX and final code.
- Dead code may appear absent because it was eliminated before PTX emission.
- Source arithmetic may contract into fused operations unless compilation semantics prohibit it.
- PTX containing a higher ISA version than the installed driver accepts can fail even when the GPU hardware is capable.
- Dynamic parallelism, device linking, and external device functions add module/linking requirements beyond a single `.entry`.

11. How to Explain in Interview

PTX is NVIDIA's typed virtual GPU instruction set. CUDA device code and other GPU front ends can lower to PTX, and then `ptxas` or the driver JIT converts PTX into architecture-specific SASS. PTX is useful as a portability and compiler boundary, but it is not normally the code the hardware directly executes. I inspect PTX for memory spaces and lowering decisions, then inspect SASS and profile to understand actual performance.

12. Quick Revision Notes

- PTX = virtual ISA; SASS = native ISA.
- `compute_xy` selects a virtual feature set; `sm_xy` selects a real code target.
- PTX registers are virtual; final allocation and spilling happen later.
- `.global`, `.shared`, `.local`, `.const`, and `.param` express address spaces.
- Predication guards lane-level execution; it does not make work free.
- PTX helps forward compatibility only with a compatible driver/PTX version.
- Interview trap: local memory is per-thread, not necessarily physically local.

13. Practice Tasks

1. Compile a small kernel to PTX with `nvcc -ptx` and locate `.entry`, `%tid.x`, `ld.global`, and `st.global`.
2. Compile `a * b + c` with and without settings that affect fused multiply-add; compare the PTX.
3. Add a thread-local fixed-size array, vary its indexing, and inspect whether local-memory operations appear.
4. Write a branch with a tiny body and a large body; compare predication and branches.
5. Use `nvcc --keep` on a small translation unit and map generated artifacts to compilation stages.
6. Explain why a PTX file generated for a new virtual architecture might fail on an older driver.

14. Final Cheat Sheet

Item	Remember
Core definition	NVIDIA virtual GPU ISA and compiler IR
Why it matters	Portability, JIT, and inspection boundary
Most asked	PTX vs SASS; <code>compute_xy</code> vs <code>sm_xy</code> ; virtual registers
Key comparison	Portable virtual code versus architecture-specific native code

Item	Remember
Biggest trap	PTX is not normally executed directly
One-line answer	"PTX is the virtual ISA that NVIDIA's backend lowers into GPU-specific SASS."

SASS

1. Overview

SASS is the informal name commonly used for NVIDIA GPU native machine instructions and their assembly-like disassembly. It is the architecture-specific code encoded in a cubin and executed by a Streaming Multiprocessor (SM).

SASS matters when source or PTX does not explain performance. It reveals the actual loads, stores, arithmetic operations, branches, register operands, predicate use, instruction modifiers, and target-dependent instruction selection. GPU library engineers use SASS when tuning hot kernels, validating tensor-core instructions, identifying spills, and understanding stalls reported by profilers.

Real systems normally generate SASS through `ptxas` during offline compilation or through the driver when loading PTX. Interviewers ask about it to determine whether a candidate can distinguish high-level intent from actual machine execution and can connect compiler output to GPU architecture.

2. Core Idea

If CUDA C++ is a program written for humans and PTX is a portable plan, SASS is the exact instruction stream for a chosen GPU target.

Conceptually, a vector operation may become instructions like:

```
S2R      R0, SR_TID.X      // read a special hardware register
IMAD     R1, R_block, c, R0 // integer index arithmetic
LDG.E    R4, [R_x]         // global-memory load
FFMA     R6, R4, R_a, R_y   // fused float multiply-add
STG.E    [R_out], R6       // global-memory store
EXIT                                // end thread
```

Exact mnemonics and encodings vary by architecture. The key reasoning steps are:

1. Find the native code for the kernel.
2. Identify memory instructions and their widths/address patterns.
3. Identify compute instructions and specialized units they use.
4. Check branches, predicates, and loop shape.
5. Check registers, spills, and occupancy constraints.
6. Correlate static instructions with dynamic profiler metrics; disassembly alone cannot tell how often a path executes or how long memory takes.

3. Important Subtopics

3.1 Cubin, fatbin, and disassembly

A **cubin** contains native code for a target architecture. A **fat binary** can bundle multiple device images. `cuobjdump` extracts embedded device code, while `nvdiasm` disassembles cubins.

- **Why it matters:** You need the correct image before interpreting native code.
- **Example:** `cuobjdump --dump-sass app` shows embedded native kernel disassembly.
- **Interview angle:** Explain that host executables may embed multiple GPU code objects.

3.2 Instruction selection

The backend chooses native instructions based on data types, target features, and optimization context.

- **Why it matters:** A high-level expression may become a fused instruction, specialized matrix instruction, or a multi-instruction sequence.
- **Example:** `a*b+c` may become an **FFMA** when contraction is legal.
- **Interview angle:** There is no guaranteed one-to-one mapping from CUDA or PTX operations to SASS.

3.3 Register allocation and spills

SASS names physical registers. If live state exceeds an allocation decision or limit, the backend emits loads/stores to thread-local storage.

- **Why it matters:** More registers can reduce occupancy; spills add memory traffic. Neither automatically implies poor performance without measurement.
- **Example:** Compiler resource output may report registers and stack/spill bytes per thread.
- **Interview angle:** Describe the trade-off between instruction-level work kept in registers and resident warps.

3.4 Scheduling, dependencies, and stalls

Machine instructions have dependencies and latency. Modern encodings include control/scheduling information that the hardware uses with warp scheduling.

- **Why it matters:** A kernel can have the right instruction count but stall on dependency chains or memory.
- **Example:** Several independent accumulators can expose instruction-level parallelism compared with one long dependent accumulation.
- **Interview angle:** Static SASS suggests dependency structure; a profiler measures actual stall reasons.

3.5 Predicate registers and branches

Native predicates conditionally enable instructions and branches at lane granularity.

- **Why it matters:** Warp divergence changes which lanes are active along paths.
- **Example:** A bounds check often sets a predicate and exits or skips work for out-of-range lanes.
- **Interview angle:** Separate predication, branch divergence, and reconvergence.

3.6 Specialized instructions

SASS exposes target-specific operations for matrix math, asynchronous copies, warp-level movement, integer dot products, and more.

- **Why it matters:** Specialized hardware is valuable only if code generation actually selects it.
- **Example:** Tensor-core kernels contain architecture-specific matrix multiply-accumulate instruction forms.
- **Interview angle:** Verify use through disassembly and profiler counters, not source API names alone.

3.7 Source correlation

Line information can let tools correlate source, PTX, and SASS.

- **Why it matters:** It connects a costly instruction region to the originating code.
- **Example:** Build with line information suitable for profiling while retaining optimization.
- **Interview angle:** Full device debugging can alter optimization and performance; line information is often the lighter choice.

4. Real-World Example

A matrix multiplication kernel is slower than expected even though the source uses a tensor-core API. The engineer:

1. Profiles the kernel and sees low tensor-pipe utilization.
2. Disassembles the cubin for the actual target GPU.
3. Finds ordinary floating-point instructions instead of the expected matrix instructions.
4. Traces this to an unsupported shape/alignment/type combination that forced a fallback path.
5. Fixes the data layout and launch conditions.
6. Rechecks SASS to confirm specialized instructions, then profiles again to confirm runtime improvement.

SASS answered **what was generated**; profiling answered **whether it ran efficiently**.

5. Diagrams / Mental Models

```

Source line:      out[i] = a * x[i] + y[i]
                  |
PTX intent:       loads + mul/add or fma + store
                  |
Native lowering:  LDG -> LDG -> FFMA -> STG
                  |
Runtime reality:  cache hits/misses, active lanes, stalls, issue rate

```

Evidence source	Answers	Does not fully answer
CUDA source	Programmer intent	Actual instruction selection
PTX	Virtual lowering	Physical registers and exact target code
SASS	Native instructions	Dynamic frequency and latency
Profiler	Runtime behavior and bottlenecks	Original design intent

6. Common Interview Questions

Q1. What is SASS?

Answer: NVIDIA GPU architecture-specific native machine code, commonly viewed as assembly-like disassembly from a cubin. **Expected:** It is the hardware-executed layer. **Common mistake:** Calling PTX and SASS interchangeable names.

Q2. Who generates SASS?

Answer: NVIDIA's native backend—`ptxas` during offline compilation or the driver backend during PTX JIT. **Expected:** Mention both paths. **Common mistake:** Saying `nvcc` alone is a single compiler that directly performs every stage.

Q3. How do you inspect SASS?

Answer: Extract or dump code with tools such as `cuobjdump`, or disassemble a cubin with `nvdiasm`; profiler source views may also correlate it. **Expected:** Choose the target image actually loaded. **Common mistake:** Inspecting PTX with `-ptx` and calling it SASS.

Q4. Why can one PTX instruction produce several SASS instructions?

Answer: PTX expresses virtual semantics; the target backend may need address setup, conversions, emulation, or multiple native operations. It can also fuse multiple PTX operations into one native instruction. **Expected:** Many-to-many lowering. **Common mistake:** Assuming line-by-line correspondence.

Q5. What does an FMA-like instruction mean?

Answer: It performs floating-point multiply-add as a fused operation with one final rounding, subject to its modifiers and compilation semantics. **Expected:** Performance plus numerical-semantic implication. **Common mistake:** Assuming separate multiply and add always have the same rounding.

Q6. How do spills appear at the machine-code level?

Answer: As loads and stores involving thread-local/stack addressing, alongside compiler resource reports showing spill bytes or stack usage. **Expected:** Correlate disassembly with compiler report. **Common mistake:** Assuming every local-memory access is definitely a register spill; explicit local arrays can also cause it.

Q7. Does a higher register count always make a kernel slower?

Answer: No. It may lower occupancy, but it can also avoid spills and preserve reuse. Performance depends on whether occupancy is limiting and whether extra registers reduce costly work. **Expected:** Trade-off, then measure. **Common mistake:** Optimizing for minimum register count as an absolute goal.

Q8. Can SASS alone identify a memory bottleneck?

Answer: It shows memory instructions and dependency structure, but not runtime hit rates, traffic, latency, or execution frequency. Use profiler metrics. **Expected:** Static versus dynamic evidence. **Common mistake:** Inferring bandwidth saturation from the mere presence of many loads.

Q9. Why is SASS architecture-specific?

Answer: GPU generations differ in instruction encoding, supported operations, pipelines, register/control details, and scheduling behavior. **Expected:** Native ISA tracks hardware. **Common mistake:** Expecting an `sm_80` cubin to run natively on arbitrary newer or older targets.

Q10. What should you inspect first in a slow kernel's SASS?

Answer: Start from a profiler hypothesis, then inspect relevant memory operations, instruction selection, branches/predicates, dependency chains, and spills. **Expected:** Evidence-driven inspection. **Common mistake:** Reading the entire disassembly without a bottleneck question.

7. Deep-Dive Questions

1. Why might two builds from the same PTX yield different native code?

They may use different driver/toolchain backend versions, optimization settings, or target architectures. Backend heuristics and instruction availability can change while preserving program semantics.

2. How does machine-code inspection confirm tensor-core use?

Find the architecture-appropriate matrix operation instructions and corroborate with runtime metrics for the relevant tensor/matrix pipelines. A dormant fallback or rarely executed path in the binary is not proof of hot-path usage.

3. What is the difference between occupancy and issue efficiency?

Occupancy measures resident warps relative to the hardware maximum. Issue efficiency concerns how effectively eligible warps issue instructions. High occupancy can still stall; lower occupancy can be enough if latency is hidden and pipelines stay busy.

4. Can instruction count predict performance?

Only weakly in isolation. Instruction mix, dependencies, active lanes, cache behavior, memory transactions, pipeline throughput, and executed loop counts matter. A shorter sequence can be slower if it creates a longer dependency chain.

5. Why can debugging builds mislead SASS analysis?

Debug options may suppress optimization, preserve variables, increase register/stack use, and change control flow. Analyze an optimized build with suitable line information for performance questions.

8. Comparison Tables

Aspect	Offline SASS	Driver-JIT SASS
Generated	Build/install time	Module load or first use
Input commonly	Device IR/PTX in toolchain	Embedded PTX
Startup cost	Low for code generation	May incur JIT latency
Target knowledge	Toolkit-supported targets at build time	Actual GPU known at deployment

Aspect	Offline SASS	Driver-JIT SASS
Reproducibility	Easier to pin with toolkit/build artifact	Can vary with installed driver
Future GPU path	Needs compatible native image or fallback	PTX may be compiled for later GPU
Static clue	Possible interpretation	Verification
Local loads/stores	Spill or explicit local object	Compiler report and source correlation
Long dependent chain	Latency exposure	Stall/dependency metrics
Specialized matrix op	Tensor hardware selected	Tensor-pipe utilization
Many branch targets	Complex/divergent control	Branch and active-lane metrics

9. Common Mistakes

- Inspecting the wrong architecture image from a fat binary.
- Treating SASS mnemonics as stable across every GPU generation.
- Inferring runtime bottlenecks from static code alone.
- Assuming more registers or lower occupancy is automatically bad.
- Calling every local-memory instruction a spill.
- Confirming that a specialized instruction exists but not whether the hot path executes it.
- Comparing debug-build disassembly with release performance.

10. Edge Cases / Special Cases

- A binary can contain multiple implementations, including fallbacks; only runtime dispatch reveals the chosen path.
- Link-time optimization and device linking can substantially change function boundaries and inlining.
- Instructions may use uniform registers, predicate registers, or specialized operand forms that differ across architectures.
- The driver may JIT PTX even when other cubins are present if no embedded native image is compatible.
- Function calls, stack frames, dynamic indexing, and recursion-capable device code can introduce local-memory/stack operations unrelated to simple spilling.

11. How to Explain in Interview

SASS is the native, architecture-specific instruction stream an NVIDIA GPU executes. It is produced offline by `ptxas` or at load time by the driver JIT. I inspect it when I need to verify actual instruction selection, register use, spills, branches, or specialized hardware operations, but I combine it with profiler data because disassembly is static and performance is dynamic.

12. Quick Revision Notes

- SASS is native; PTX is virtual.
- Cubins contain native device code; fat binaries can contain several images.
- `cuobjdump` and `nvdiasm` help inspect native code.
- Register count trades occupancy against retained values/spills.
- Static code does not reveal cache misses, path frequency, or stall duration.

- Verify specialized hardware with both instructions and profiler counters.
- Interview trap: an instruction present in the binary may not be on the executed hot path.

13. Practice Tasks

1. Compile a kernel for one `sm_xy`, dump SASS, and identify its load, compute, store, and exit instructions.
2. Compare release and device-debug builds and record changes in register/stack use.
3. Create two versions of an accumulation loop—one accumulator versus several—and compare dependency structure and runtime.
4. Force a thread-local array with dynamic indexing and inspect local-memory operations.
5. Compile for two supported SM architectures and compare native instruction selection.
6. Use profiler metrics to validate one hypothesis formed from disassembly.

14. Final Cheat Sheet

Item	Remember
Core definition	NVIDIA architecture-specific native GPU instructions
Why it matters	Shows what the hardware actually receives
Most asked	SASS vs PTX; registers/spills; static vs dynamic analysis
Key comparison	Native cubin at build time versus native code from driver JIT
Biggest trap	SASS alone cannot prove the runtime bottleneck
One-line answer	"SASS is the GPU-specific machine code produced from PTX or lower compiler IR and executed by the SM."

JIT Compilation

1. Overview

Just-In-Time (JIT) compilation generates or specializes executable code during program execution, module loading, or first use instead of producing every final machine-code variant ahead of time. In the NVIDIA GPU stack, the most direct example is the CUDA driver compiling embedded PTX into native code for the current GPU. Frameworks also JIT-compile tensor programs into kernels specialized for observed shapes, data types, layouts, constants, and hardware.

JIT matters because runtime knows facts that an ahead-of-time compiler may not know: the exact GPU, input shape, strides, constant values, and executed graph path. Specialization can remove generality and unlock fusion, constant folding, better launch geometry, and hardware-specific code generation. Its costs are compilation latency, cache management, warm-up behavior, and less predictable first-request performance.

Real systems use JIT in deep-learning frameworks, database query engines, scientific DSLs, shader systems, dynamic language runtimes, and user-generated kernel services. Interviewers ask about JIT to test whether

candidates understand the latency-throughput trade-off, cache keys, invalidation, specialization, and the boundary between PTX driver JIT and framework-level graph/kernel JIT.

2. Core Idea

Ahead-of-time compilation prepares meals before customers arrive; JIT waits for the order, learns the exact ingredients and portion size, then cooks a specialized meal. Waiting costs time once, but repeated identical orders can reuse the result.

```

First call with signature S
    |
capture/trace or choose PTX
    |
optimize and generate target code
    |
cache under key K(S, target, compiler, options)
    |
execute

Later call with same compatible key -> cache hit -> execute
Different shape/dtype/target          -> cache miss -> compile another variant

```

Small example: an elementwise kernel written generically for arbitrary strides receives a contiguous `float32` tensor of length divisible by a vector width. A JIT can specialize guards for those facts, use vector loads, fold constants, and omit generic stride branches. If a later input violates the guards, the runtime must select a fallback or compile a new variant.

3. Important Subtopics

3.1 Driver PTX JIT

The CUDA driver can translate PTX in a module into native instructions for the active GPU.

- **Why it matters:** It provides a target-at-deployment path and can support compatible GPUs not known when the application was built.
- **Example:** A fat binary lacks a native image for the current GPU but contains compatible PTX.
- **Interview angle:** This JIT lowers one device module; it does not automatically perform framework graph fusion.

3.2 Runtime source compilation

Runtime compilation APIs can compile CUDA-like device source or generated source into PTX or other device code while the application runs.

- **Why it matters:** Applications can generate kernels from user expressions, schemas, or models.
- **Example:** A simulation DSL emits a kernel customized for the number of fields.
- **Interview angle:** Source-to-PTX runtime compilation and driver PTX-to-native JIT are distinct stages that may both occur.

3.3 Tracing, scripting, and capture

A framework must first obtain a compilable program representation.

- **Tracing:** Records operations observed for example inputs; data-dependent paths may be missed.
- **Scripting/bytecode analysis:** Represents control flow from program semantics rather than one execution.
- **Graph capture:** Intercepts framework operations into a graph with guards.
- **Why it matters:** An incorrect captured graph produces incorrect specialization no matter how good code generation is.
- **Interview angle:** Explain graph breaks and dynamic control flow.

3.4 Specialization and guards

The compiler assumes facts such as dtype, rank, shape relations, strides, alignment, or constants and installs runtime checks.

- **Why it matters:** Stronger facts produce faster code but more variants and recompilation.
- **Example:** Specialize on rank and contiguity while keeping batch size symbolic.
- **Interview angle:** A correct JIT must guard every assumption that can affect semantics.

3.5 Cache keys and invalidation

A JIT cache key may include program identity, input signature, target GPU, compiler/driver version, flags, and linked-library state.

- **Why it matters:** An incomplete key can reuse incompatible code; an overly specific key causes cache explosion.
- **Example:** Native code for one target architecture cannot blindly serve another.
- **Interview angle:** Cache correctness comes before cache hit rate.

3.6 Warm-up and compilation latency

First execution includes capture, optimization, code generation, assembly, linking/loading, or autotuning.

- **Why it matters:** Interactive services care about tail latency even if steady-state throughput improves.
- **Example:** A model server warms common shapes before accepting traffic.
- **Interview angle:** Report cold and warm performance separately.

3.7 Dynamic shapes

Dynamic dimensions can be treated as fully static variants, bounded/symbolic values, or general runtime values.

- **Why it matters:** Static specialization is fast but can recompile for every size; symbolic compilation reuses code but restricts optimization.
- **Example:** One kernel supports any batch size satisfying a guard while specializing fixed head dimension.
- **Interview angle:** Choose specialization boundaries based on workload distribution.

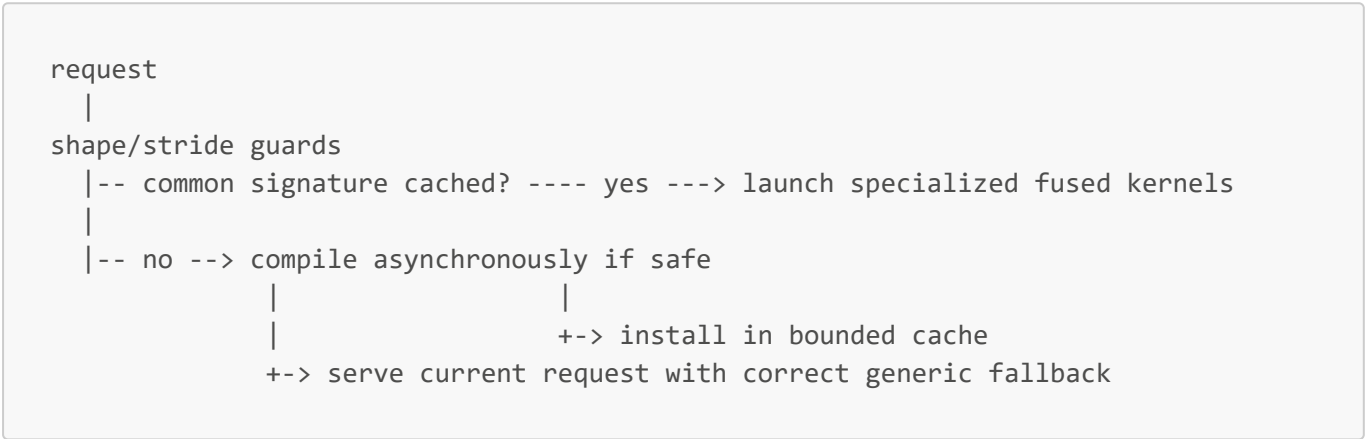
3.8 Fallbacks and deoptimization

When guards fail or an operation is unsupported, the system can execute eager code, call a library kernel, break the graph, or compile a new variant.

- **Why it matters:** Production correctness cannot depend on inputs always matching an optimistic profile.
- **Example:** A non-contiguous tensor takes a generic fallback.
- **Interview angle:** Deoptimization must preserve ordering, side effects, and state.

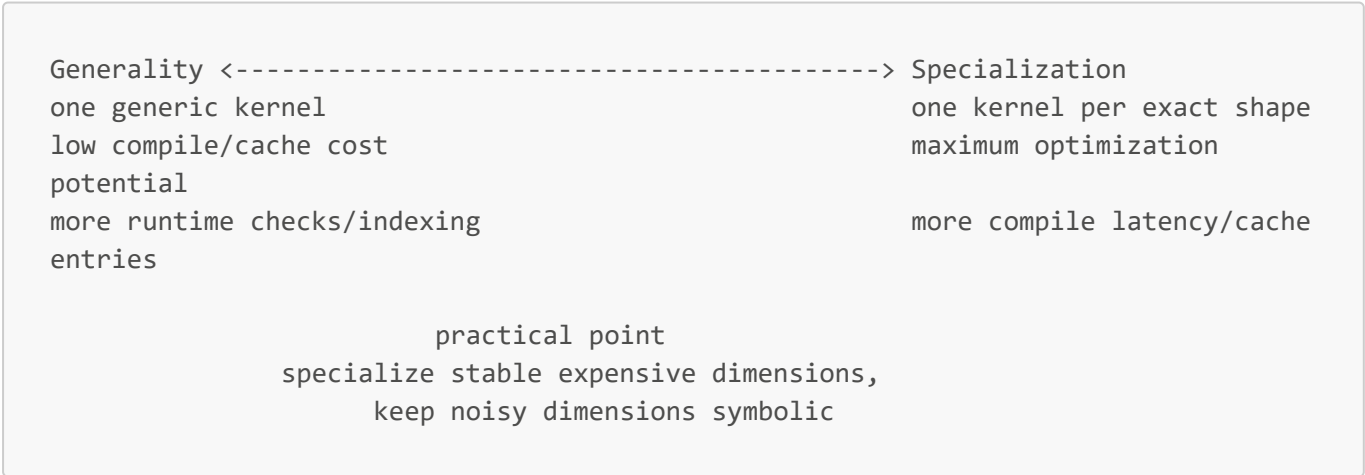
4. Real-World Example

An online inference service receives mostly batch sizes 1, 8, and 32, with occasional unusual sizes. Its JIT strategy is:



The service prewarms the three common signatures during deployment. A bounded cache prevents one-off shapes from consuming unlimited memory. Metrics separate compile time, cache hits, fallback rate, and execution time. This design makes the JIT operationally observable rather than hiding compilation inside unexplained latency spikes.

5. Diagrams / Mental Models



Phase	Cold call	Warm call
Capture/guards	Build or validate	Validate
Optimization	Run passes	Reuse result
Code generation	Generate/assemble	None on hit

Phase	Cold call	Warm call
Load	Load module	Already loaded or quick lookup
Execute	Run kernel	Run kernel

6. Common Interview Questions

Q1. What is JIT compilation?

Answer: Compilation or specialization performed while a program runs or loads, using runtime information to create executable code. **Expected:** Contrast with AOT. **Common mistake:** Calling any dynamic function dispatch JIT compilation.

Q2. What is CUDA driver JIT?

Answer: The driver translates PTX into native code for the active NVIDIA GPU when a compatible native image is unavailable or PTX loading is requested. **Expected:** PTX-to-SASS/native path. **Common mistake:** Saying it compiles the entire CUDA host application.

Q3. Why can JIT code be faster than AOT code?

Answer: Runtime values enable specialization, constant folding, layout decisions, fusion, tuned launch parameters, and exact-target code generation. **Expected:** State that improvement is possible, not guaranteed. **Common mistake:** Ignoring compile time or assuming JIT always wins.

Q4. What should be in a JIT cache key?

Answer: All facts that affect generated code or validity: program/version, guarded input properties, target, compiler/options, and relevant linked/runtime state. **Expected:** Correctness and granularity. **Common mistake:** Keying only by tensor shape.

Q5. What is a guard?

Answer: A runtime check proving that assumptions used by a specialized compiled variant hold for the current input/state. **Expected:** Guard failure selects recompile or fallback. **Common mistake:** Treating guards only as performance hints.

Q6. What is cache explosion?

Answer: Excessive compiled variants caused by over-specialization or highly variable inputs, increasing compilation, memory, and lookup costs. **Expected:** Bound cache and use symbolic/general variants. **Common mistake:** Assuming every distinct input deserves a kernel.

Q7. What is a graph break?

Answer: A boundary where capture/compilation cannot continue, so execution returns to eager/runtime handling before compilation may resume. **Expected:** It reduces optimization scope and may add overhead. **Common mistake:** Saying every graph break changes semantics.

Q8. How do you benchmark a JIT system?

Answer: Measure cold compile-plus-run latency, warm execution latency, cache hit rate, number/size of variants, fallback rate, and end-to-end workload throughput. **Expected:** Synchronize appropriately for GPU timing. **Common mistake:** Timing asynchronous kernel launch without measuring completion.

Q9. How do dynamic shapes affect JIT compilation?

Answer: They force a choice between exact specialization, symbolic/bounded code, or generic fallback. Exact variants optimize more but can repeatedly compile. **Expected:** Explain guards and workload distribution. **Common mistake:** Calling dynamic shapes impossible to compile.

Q10. What happens when a guard fails?

Answer: The runtime must not execute the invalid variant; it recompiles for the new case, uses another cached variant, or follows a correct fallback/deoptimization path. **Expected:** Correctness first. **Common mistake:** Ignoring state and side effects around fallback.

7. Deep-Dive Questions

1. How would you prevent a compilation stampede?

Deduplicate in-flight compilation by cache key so one worker compiles and others wait or use a fallback. Bound queues and cache size; prewarm common variants. Avoid holding a global lock during expensive compilation if concurrency requires more throughput.

2. How can asynchronous JIT remain correct?

Execute a known-correct fallback until compilation completes, publish the compiled artifact atomically, and validate the same guards at use time. Compilation failure must leave the fallback available.

3. What makes persistent JIT caches difficult?

Artifacts depend on program hashes, compiler and driver compatibility, target architecture, flags, libraries, ABI/contracts, and sometimes environment settings. Cache entries need validation, atomic writes, eviction, and corruption handling.

4. Why can aggressive specialization reduce overall performance?

Extra compilation and cache misses can exceed kernel savings, especially for short-lived jobs or diverse shapes. Many variants consume memory and harm instruction/module locality. Optimize end-to-end cost, not only steady-state kernel time.

5. How do side effects complicate graph JIT?

Reordering, duplication, fallback, or recompilation must preserve mutations, random-number state, exceptions, I/O, and synchronization. Compilers use effect tracking, functionalization, barriers, or graph breaks to maintain semantics.

8. Comparison Tables

Aspect	AOT compilation	JIT compilation
Final code generated	Before deployment/run	At load or execution time
Runtime information	Limited/profile-guided at best	Exact observed target/input facts
First-run latency	Usually lower	Can be high
Reproducibility	Easier to pin	Depends on runtime compiler/driver/cache
Variant control	Explicit build target set	Dynamic, needs guards and eviction
Best fit	Stable targets, strict cold start	Repeated workloads with useful specialization

JIT layer	Input	Output	Main optimization scope
Driver JIT	PTX	Target-native GPU code	Device instruction lowering
Runtime source compiler	Device source/generated source	PTX or device image	Kernel source specialization
Framework graph JIT	Captured tensor graph	Fused/specialized kernels + schedule	Multi-operation graph and shapes

9. Common Mistakes

- Measuring only warm execution and hiding compilation cost.
- Using incomplete cache keys.
- Specializing every dimension/value and causing variant explosion.
- Assuming a tracing run captures all data-dependent control flow.
- Lacking a correct fallback for guard failure or compiler failure.
- Confusing driver PTX JIT with graph-level framework compilation.
- Ignoring GPU asynchrony in timing.
- Letting untrusted generated source/options cross a compilation trust boundary without isolation and limits.

10. Edge Cases / Special Cases

- Cache hits may still require module loading into a new process/context.
- Forked processes, containers, read-only filesystems, and multi-user caches complicate persistent caching.
- Random operations and mutable state require explicit effect handling.
- Shape equality is insufficient when strides, alignment, dtype, device, or broadcasting affect code.
- Driver upgrades can legitimately invalidate or change native-code cache entries.
- Compilation can fail due to resource limits, unsupported operations, or timeouts; production systems need bounded fallback behavior.

11. How to Explain in Interview

JIT compilation creates code at load or run time so the compiler can specialize for the actual GPU and inputs. CUDA's driver JIT lowers PTX to native code, while framework JITs may also capture graphs, fuse operations, and generate kernels. The performance win must repay compilation and guard overhead, so a good system uses correct guards, complete cache keys, bounded variants, prewarming for common cases, and a reliable fallback.

12. Quick Revision Notes

- JIT uses runtime facts; AOT avoids runtime compilation cost.
- Driver JIT is PTX-to-native; graph JIT has a higher optimization scope.
- Specialization requires guards.
- Cache key must include every code-validity dependency.
- Cold latency, warm latency, hit rate, variants, and fallback rate all matter.
- Dynamic shapes are a specialization policy problem, not an automatic blocker.
- Interview trap: do not benchmark asynchronous launch as completed GPU execution.

13. Practice Tasks

1. Build a PTX-only kernel path and measure first-load versus repeated-load latency.
2. Design a cache key for a kernel specialized by dtype, rank, contiguity, vector width, target, and compiler flags.
3. Implement a tiny expression-to-CUDA-source generator with two guarded variants and one generic fallback.
4. Simulate requests with a long-tailed shape distribution and compare exact-shape versus symbolic-shape cache policies.
5. Write a benchmark that reports compile time separately from synchronized execution time.
6. Explain how you would safely publish a compiled artifact when several workers request it concurrently.

14. Final Cheat Sheet

Item	Remember
Core definition	Runtime/load-time generation or specialization of executable code
Why it matters	Uses actual target and input facts for better code
Most asked	JIT vs AOT; guards; cache keys; warm-up; dynamic shapes
Key comparison	Driver PTX JIT versus framework graph/kernel JIT
Biggest trap	Warm kernel speed can hide unacceptable compilation and cache costs
One-line answer	"JIT trades compilation and cache complexity for runtime specialization and exact-target code generation."

Kernel Fusion

1. Overview

Kernel fusion combines work that would otherwise run as multiple GPU kernels into one kernel, or into fewer kernels. The primary goals are to reduce launch overhead and avoid writing intermediate values to global memory only to read them back immediately.

Fusion matters because many tensor operations perform little arithmetic per element. Launching `add`, then `bias`, then `ReLU` as three kernels can be dominated by launch and memory traffic. A fused kernel can load inputs once, compute the chain in registers, and store only the final output.

Real systems use fusion in deep-learning compilers, inference engines, image pipelines, database expression evaluators, and scientific array frameworks. Interviewers ask about fusion because it joins compiler dependence analysis with GPU performance reasoning: fusion is beneficial only when legality is preserved and resource costs do not outweigh saved traffic and launches.

2. Core Idea

Without fusion:

```
Kernel 1: t1 = x + bias      x,bias -> GPU cores -> t1 in global memory
Kernel 2: t2 = relu(t1)    t1      -> GPU cores -> t2 in global memory
Kernel 3: y  = t2 * scale   t2      -> GPU cores -> y  in global memory
```

With fusion:

```
One kernel per element:
  load x and bias
  v = x + bias
  v = max(v, 0)
  v = v * scale
  store y
```

Step by step:

1. Build producer-consumer relationships between operations.
2. Prove that combining them preserves dependencies, indexing, effects, and synchronization semantics.
3. Choose a fused iteration domain and thread/block mapping.
4. Keep intermediate values in registers or shared memory rather than materializing global tensors.
5. Generate one kernel and compare saved memory/launch cost against added registers, code size, synchronization, and reduced parallelism.

A fused kernel is not merely source concatenation. The compiler must reconcile shapes, broadcasting, reductions, layouts, and execution schedules.

3. Important Subtopics

3.1 Vertical producer-consumer fusion

A producer is fused into its consumer so intermediate values stay near the thread that uses them.

- **Why it matters:** Eliminates intermediate global-memory round trips.
- **Example:** Bias add → activation → scale.
- **Interview angle:** Best when producer and consumer have compatible iteration/indexing domains.

3.2 Horizontal fusion

Independent operations with compatible execution shapes share one launch.

- **Why it matters:** Reduces launch overhead and may share inputs.
- **Example:** Compute sum and sum-of-squares in one pass.
- **Interview angle:** It can improve input reuse but increase register use or complicate output scheduling.

3.3 Epilogue fusion

An expensive core operator includes following elementwise work in its output stage.

- **Why it matters:** Matrix multiplication or convolution already has result fragments in registers; applying bias/activation before store avoids another read/write cycle.
- **Example:** GEMM + bias + GELU epilogue.
- **Interview angle:** Common in optimized libraries and inference compilers.

3.4 Reduction fusion

Elementwise producers may fuse into a reduction, and limited consumers may fuse after it.

- **Why it matters:** Producer values need not be materialized, but the reduction introduces cross-thread communication.
- **Example:** Square each element then reduce for an L2 norm.
- **Interview angle:** Reduction boundaries, synchronization, and multi-stage reductions make fusion harder.

3.5 Fusion legality

Dependence, alias, effect, and memory-order analysis determines whether reordering/combining operations preserves results.

- **Why it matters:** A fast incorrect kernel is useless.
- **Example:** In-place mutation or overlapping views may prevent apparently obvious fusion.
- **Interview angle:** Mention random/stateful operations, exceptions, and synchronization effects.

3.6 Fusion profitability

Legal fusion may still be slower.

- **Why it matters:** A larger kernel may use more registers/shared memory, lower occupancy, duplicate computation, reduce scheduling flexibility, or exceed instruction-cache comfort.
- **Example:** Fusing two large consumers may duplicate a costly producer for each consumer.
- **Interview angle:** Use a cost model and profiling, not "fuse everything."

3.7 Shape, layout, and broadcasting

Fused operations need a compatible indexing plan.

- **Why it matters:** Broadcast and non-contiguous layouts can add expensive address calculation or force unfavorable access patterns.
- **Example:** Row-wise reduction followed by broadcast normalization needs mapping from output rows back to all elements.
- **Interview angle:** Same logical shape does not guarantee same physical layout.

3.8 Persistent and mega-kernels

Some designs keep work/resident state in a long-lived or very large kernel.

- **Why it matters:** They reduce launches and global traffic but can limit occupancy and inter-kernel scheduling.
- **Example:** A fused attention implementation performs several algorithmic phases within one coordinated kernel design.
- **Interview angle:** Distinguish ordinary elementwise fusion from algorithmically redesigned fused kernels.

4. Real-World Example

Consider layer normalization:

```
x -> compute mean -> compute variance -> normalize -> scale -> bias -> y
```

A naïve implementation launches several kernels and materializes mean, centered values, variance, and normalized values. A fused implementation can assign a row to a block:

1. Threads load row elements.
2. A block reduction computes mean and variance using a numerically acceptable algorithm.
3. Threads normalize their elements.
4. Scale and bias are applied in the epilogue.
5. Only the final output and required saved statistics are stored.

The fused design saves launches and memory traffic, but row width controls shared memory, reduction strategy, occupancy, and numerical behavior. Very wide rows may require multiple passes or a different schedule.

5. Diagrams / Mental Models

Unfused bytes (conceptual):

```
input -> [K1] -> temp1 -> [K2] -> temp2 -> [K3] -> output
           write/read      write/read
```

Fused bytes:

```
input -> [ K1 + K2 + K3, intermediates in registers ] -> output
```

Fusion benefit	Fusion cost/risk
Fewer launches	Longer compilation
Fewer global reads/writes	Higher register pressure
Producer-consumer locality	More shared memory/synchronization
More cross-op optimization	Larger code/instruction footprint
Shared indexing/load work	Reduced scheduling flexibility
Better epilogue reuse	Possible recomputation or poor layout compromise

6. Common Interview Questions

Q1. What is kernel fusion?

Answer: Combining operations that would run as separate GPU kernels into one or fewer kernels while preserving semantics. **Expected:** Launch and intermediate-memory savings. **Common mistake:** Defining it only as concatenating source functions.

Q2. Why does fusion improve GPU performance?

Answer: It can eliminate kernel-launch overhead and global-memory materialization of intermediates, improve locality, and expose cross-operation simplification. **Expected:** Especially useful for memory-bound elementwise chains. **Common mistake:** Claiming it reduces the arithmetic required in every case.

Q3. When can fusion hurt performance?

Answer: When the fused kernel increases registers/shared memory, lowers useful occupancy, duplicates computation, creates worse access patterns, grows code too much, or removes concurrent scheduling opportunities. **Expected:** Profitability versus legality. **Common mistake:** "More fusion is always better."

Q4. What is producer-consumer fusion?

Answer: Computing a producer's value inside its consumer kernel so the intermediate is not globally materialized. **Expected:** Compatible indexing/dependency. **Common mistake:** Ignoring multiple consumers that may require reuse or recomputation.

Q5. What is horizontal fusion?

Answer: Combining independent operations, commonly sharing an iteration domain or inputs, into one launch. **Expected:** Launch/input reuse. **Common mistake:** Describing it as a dependency chain.

Q6. What is epilogue fusion?

Answer: Applying follow-up operations such as bias, activation, scaling, or quantization while a core operator's output is still in registers/fragments before its final store. **Expected:** GEMM/convolution example. **Common mistake:** Assuming arbitrary later reductions fit an epilogue.

Q7. What makes fusion legal?

Answer: Dependencies, aliases, side effects, memory ordering, shapes, and execution semantics must remain equivalent after combining/reordering. **Expected:** Correctness proof/analysis. **Common mistake:** Checking only tensor shapes.

Q8. Why are reductions harder to fuse?

Answer: They change the iteration domain and require communication/synchronization across elements or threads. Following operations may need the completed reduction value. **Expected:** Map/reduce boundary. **Common mistake:** Treating every operation as independent per element.

Q9. How do you decide whether to fuse?

Answer: Use legality analysis plus a profitability model considering bytes saved, launches removed, compute duplication, resource use, parallelism, layout, and expected shapes; then validate by profiling. **Expected:** Cost model. **Common mistake:** Looking only at kernel count.

Q10. Does fusion always eliminate intermediate storage?

Answer: No. Some intermediates may need shared memory, registers, partial global storage, or materialization for multiple consumers, backward passes, aliasing, or size/resource reasons. **Expected:** Fusion can be partial. **Common mistake:** Assuming one kernel means zero intermediate state.

7. Deep-Dive Questions

1. How does fusion interact with occupancy?

Combining live values enlarges register live ranges and may require more shared memory. Fewer blocks/warps may reside per SM. This matters only if reduced residency fails to hide latency or limits throughput; use resource reports and profiler evidence.

2. How should a compiler handle a producer with multiple consumers?

It can materialize once, fuse with one consumer and materialize for another, duplicate cheap computation, or fuse all compatible consumers horizontally. The choice depends on producer cost, output size, consumer layouts, and resource pressure.

3. Why can numerical results change after legal-looking fusion?

Reassociation, contraction, reduction order, precision changes, and removal of intermediate rounding can change floating-point results. The compiler must obey the selected numerical contract and tolerance, not assume real-number algebra.

4. How does fusion affect autograd/training?

Backward may need forward intermediates. A compiler may save them, recompute them, or generate a joint forward/backward strategy. Saving costs memory; recomputation costs compute; in-place fusion can violate versioning/alias rules.

5. What is the relationship between tiling and fusion?

Fusion decides which operations share a kernel/scope; tiling decides how iteration domains and data are partitioned for threads, blocks, caches, and shared memory. A profitable fusion often requires a compatible tile schedule, not only a graph merge.

8. Comparison Tables

Type	Relationship	Main win	Main risk
Vertical	Producer -> consumer	Remove intermediate traffic	Incompatible mapping or multiple consumers
Horizontal	Independent siblings	Fewer launches/shared input	Register/output pressure
Epilogue	Core op -> simple consumers	Store only final result	Limited epilogue expressiveness/resources
Reduction fusion	Map/reduce/consumer	Avoid producer/partial tensors	Synchronization and domain change
Aspect	Unfused kernels	Fused kernel	
Launches	More	Fewer	
Intermediate global traffic	Usually more	Often less	
Per-kernel resources	Smaller	Potentially larger	
Scheduling flexibility	Higher between kernels	Lower inside fused unit	
Compilation complexity	Lower	Higher	
Debug/profiling isolation	Easier	Source attribution can be harder	

9. Common Mistakes

- Assuming every adjacent graph node should fuse.
- Counting kernel launches but ignoring bytes and resource pressure.
- Ignoring aliasing, mutation, random state, and synchronization.
- Assuming logical shape compatibility implies efficient physical layout.
- Forgetting that training may need forward intermediates.
- Duplicating an expensive producer to avoid one temporary.
- Comparing kernels without end-to-end synchronized timing.
- Calling a large hand-designed algorithm merely an elementwise fused kernel.

10. Edge Cases / Special Cases

- Zero-sized tensors may bypass kernels or require guard-safe launch logic.
- Very small workloads can be launch-bound; very large workloads may be bandwidth/resource-bound.
- Broadcast stride zero, negative/irregular strides, overlapping views, and in-place updates complicate indexing and legality.
- Floating-point reduction order can change with fused schedules.

- Separate streams or explicit event dependencies may make cross-kernel scheduling semantically important.
- A library call may already implement a highly tuned fused epilogue; replacing it with generated code can regress performance.

11. How to Explain in Interview

Kernel fusion combines compatible GPU operations so intermediates remain in registers or shared memory and fewer kernels are launched. It is especially effective for memory-bound elementwise chains and epilogues. Fusion requires dependency, alias, effect, shape, and numerical correctness checks, and it is not always profitable because a larger kernel can increase registers, shared memory, code size, or recomputation and reduce occupancy.

12. Quick Revision Notes

- Main wins: fewer launches and less global-memory traffic.
- Vertical = producer-consumer; horizontal = compatible siblings; epilogue = post-op inside core kernel.
- Legality and profitability are separate decisions.
- Reductions change iteration domains and require communication.
- Watch registers, shared memory, occupancy, code size, layout, and duplicate work.
- Fusion and tiling must agree on an execution schedule.
- Interview trap: fewer kernels does not automatically mean faster execution.

13. Practice Tasks

1. Implement `bias + ReLU + scale` as three kernels and as one fused kernel; measure synchronized time and memory traffic.
2. Estimate bytes moved per output element before and after fusion.
3. Fuse square with sum reduction and explain block-level synchronization.
4. Compare a fused kernel's register count and occupancy with its unfused components.
5. Draw a graph with one expensive producer and two consumers; propose materialize, duplicate, and partial-fusion plans.
6. Test contiguous, transposed, broadcast, zero-size, and odd-length inputs for a fused elementwise kernel.

14. Final Cheat Sheet

Item	Remember
Core definition	Combine operations into fewer GPU kernels
Why it matters	Saves launch overhead and intermediate global traffic
Most asked	Why fusion helps/hurts; legality; reductions; occupancy
Key comparison	Vertical versus horizontal versus epilogue fusion
Biggest trap	Legal fusion is not necessarily profitable fusion

Item	Remember
One-line answer	"Fusion trades larger, more constrained kernels for fewer launches and less intermediate memory traffic."

Graph Compilers

1. Overview

A **graph compiler** converts a program—often a machine-learning or tensor program—into a graph of operations and then transforms, partitions, schedules, and lowers that graph into executable kernels and library calls for target hardware. Nodes represent computations; edges represent values, control dependencies, tokens/effects, or state relationships.

Graph compilers matter because optimizing each kernel independently misses whole-program opportunities. At graph scope, a compiler can propagate shapes/constants, remove unused work, fuse operators, choose layouts, reuse memory, select vendor-library implementations, schedule communication, and generate specialized kernels.

Real systems use graph compilation in training and inference frameworks, mobile runtimes, accelerator stacks, database query engines, image-processing systems, and differentiable-programming tools. Examples of relevant ecosystems include XLA, TorchInductor-related pipelines, TensorRT-style optimization, TVM, and MLIR-based compiler stacks; their exact architectures differ, but the interview concepts are shared. Interviewers ask about graph compilers to test reasoning across program semantics, IR design, shape systems, partitioning, cost models, memory planning, and target code generation.

2. Core Idea

Consider:

```
y = relu(x @ w + bias)
```

An eager runtime may dispatch matrix multiply, addition, and activation separately. A graph compiler sees the relationship:

```

x ----\
      MatMul ---> Add ---> ReLU ---> y
w ----/
      ^
      |
    bias

```

It can reason step by step:

1. **Capture/parse:** Build a graph with typed values and effects.
2. **Normalize:** Convert equivalent constructs into compiler-friendly canonical forms.

3. **Infer:** Propagate dtype, rank, shape constraints, layouts, and constants.
4. **Simplify:** Fold constants, remove dead nodes, and simplify algebra under legal numerical rules.
5. **Partition:** Decide which regions go to generated kernels, vendor libraries, CPU, or another accelerator.
6. **Fuse/schedule:** Combine compatible nodes and choose tiling, parallel mapping, and layouts.
7. **Plan memory:** Reuse buffers using liveness while respecting aliases and asynchronous execution.
8. **Lower:** Progress through tensor/loop/GPU IR to LLVM/PTX/native or invoke precompiled libraries.
9. **Guard and execute:** Check specialization assumptions, cache variants, launch work, and fall back when necessary.

The graph is a data structure for proofs and transformations, not just a picture of layer names.

3. Important Subtopics

3.1 Graph capture and representation

Capture can come from tracing, source/bytecode analysis, explicit graph APIs, or ahead-of-time export.

- **Why it matters:** Capture determines which control flow, mutations, and operations the compiler can see.
- **Example:** Tracing records only the branch taken by sample inputs unless guards or higher-level control-flow nodes represent alternatives.
- **Interview angle:** Discuss graph breaks and semantic coverage, not only performance.

3.2 Data-flow, control-flow, and effect representation

Pure data-flow edges are insufficient for mutations, randomness, I/O, communication, exceptions, and synchronization.

- **Why it matters:** Transformations must preserve ordering of observable effects.
- **Example:** Random-number generation can thread an explicit state/token dependency through nodes.
- **Interview angle:** A DAG model may need nested regions, cycles/control-flow operators, or effect tokens for full programs.

3.3 Shape and type inference

The compiler computes known dimensions, symbolic relationships, dtypes, devices, and sometimes value ranges.

- **Why it matters:** Shapes decide output allocation, legality, fusion domains, library selection, and guards.
- **Example:** Matrix multiplication requires compatible inner dimensions, while batch dimensions may broadcast.
- **Interview angle:** Distinguish static, symbolic, and data-dependent shapes.

3.4 Canonicalization and constant folding

Canonicalization rewrites many equivalent forms into a smaller set; constant folding evaluates compile-time-known expressions.

- **Why it matters:** Later passes become simpler and more predictable.
- **Example:** Transpose-of-transpose can disappear; a constant reshape shape can be resolved early.

- **Interview angle:** Folding huge constants can increase compile time or artifact size, so it also needs policy.

3.5 Operator fusion

Graph regions are grouped into fused kernels or library epilogues.

- **Why it matters:** Reduces launches and intermediate memory traffic.
- **Example:** MatMul + bias + activation becomes a tuned GEMM call with supported epilogue or a generated fused region.
- **Interview angle:** Fusion legality, profitability, layout, reductions, and resource pressure.

3.6 Layout and data-placement optimization

The compiler chooses or propagates tensor layouts and decides when transformations/copies are worthwhile.

- **Why it matters:** A locally optimal kernel layout may force expensive conversions around it.
- **Example:** Keep a blocked layout across several operators rather than convert before and after each one.
- **Interview angle:** Optimize layout globally, not node by node.

3.7 Pattern rewriting and library selection

Subgraphs can match semantic patterns and map to optimized primitives.

- **Why it matters:** Vendor libraries often outperform generated kernels for mature dense operations.
- **Example:** Recognize attention-like structure or a convolution+bias+activation supported by a backend.
- **Interview angle:** Pattern matching must account for semantics, attributes, shapes, dtypes, layouts, and numerical contracts.

3.8 Partitioning and heterogeneous execution

Unsupported or unprofitable regions may remain in the original framework or go to another device/backend.

- **Why it matters:** Transfers and synchronization at partition boundaries can erase accelerator gains.
- **Example:** Most of a graph runs on GPU, but one unsupported operator causes device-to-host-to-device copies.
- **Interview angle:** Maximize useful regions while pricing boundary cost.

3.9 Scheduling and lowering

After choosing an algorithm, the compiler maps iteration to tiles, blocks, warps, threads, vectors, and memory spaces.

- **Why it matters:** The same graph and arithmetic can yield radically different performance under different schedules.
- **Example:** Tile matrix multiplication for shared-memory reuse and map subtiles to warps.
- **Interview angle:** Algorithm describes what; schedule describes how/where/when.

3.10 Memory planning

Graph-wide liveness allows buffers to be reused after their last use.

- **Why it matters:** Peak memory often limits model size and batch size.
- **Example:** Two non-overlapping temporary tensors share one allocation arena.
- **Interview angle:** Aliases, saved tensors, streams, and asynchronous lifetime prevent naïve reuse.

3.11 Cost models and autotuning

Cost models estimate latency/resource use; autotuning empirically benchmarks candidate schedules.

- **Why it matters:** Hardware and shapes make one universal schedule unrealistic.
- **Example:** Try tile sizes from a bounded candidate set and cache the best valid result.
- **Interview angle:** Tuning cost, noise, determinism, cache keys, and production constraints.

3.12 Multi-level IR and progressive lowering

Compilers often use graph, tensor, loop, GPU, and machine-level representations.

- **Why it matters:** Each optimization runs where its required information is explicit.
- **Example:** Fuse at tensor level, tile at loop level, map to threads at GPU level, then lower to LLVM/PTX.
- **Interview angle:** Lowering too early destroys high-level structure; lowering too late can hide target constraints.

4. Real-World Example

An inference graph contains:

```
input -> linear -> bias -> GELU -> linear -> bias -> residual add -> layer norm
```

A practical compiler might:

1. Propagate fixed hidden size and symbolic batch size.
2. Fold immutable weights into deployment constants or prepack them where supported.
3. Select a vendor GEMM for each linear layer.
4. Fuse the first bias and GELU into a supported GEMM epilogue.
5. Fuse the second bias, residual add, and parts of normalization when legal/profitable, or choose a tuned normalization kernel.
6. Keep compatible layouts across the region.
7. Reuse temporary buffers whose live ranges do not overlap.
8. Generate guards for dtype, hidden size, strides, and device.
9. Cache code for common batch ranges and use a fallback for unsupported shapes.

The result is not necessarily one giant kernel. Good graph compilation chooses a small set of efficient kernels/library calls with inexpensive boundaries.

5. Diagrams / Mental Models

```
Program
|
v
```

```

[Capture IR] -- semantics/effects/control flow
|
v
[Graph IR] --- shapes, constants, fusion, partitioning, layouts
|
v
[Tensor/Loop IR] --- tiling, vectorization, parallel mapping
|
v
[GPU/LLVM IR] --- address spaces, low-level optimization
|
v
[PTX] -> [SASS] -> GPU

```

Optimization	Graph information required	Typical payoff
Dead-code elimination	Use edges and effects	Remove unnecessary work
Constant folding	Known values and semantics	Remove runtime computation
Fusion	Producer-consumer/effect/layout facts	Fewer launches and temporaries
Layout propagation	Consumer requirements and conversion costs	Fewer transposes/copies
Memory reuse	Buffer sizes, liveness, aliases, streams	Lower peak memory
Partitioning	Backend support and boundary costs	Useful accelerator regions
Autotuning	Candidate schedules and representative shapes	Better target-specific performance

6. Common Interview Questions

Q1. What is a graph compiler?

Answer: A compiler that represents a program as connected operations/values and optimizes regions across operator boundaries before lowering them to kernels, libraries, and target code. **Expected:** Whole-graph scope. **Common mistake:** Calling any neural-network runtime a graph compiler.

Q2. Why compile a graph instead of launching eager operators?

Answer: The graph exposes cross-operation opportunities: fusion, constant propagation, layout planning, buffer reuse, partitioning, and specialization. **Expected:** End-to-end optimization. **Common mistake:** Saying only that graphs reduce Python overhead.

Q3. How is a computation graph captured?

Answer: Through tracing, source/bytecode analysis, explicit graph construction, export, or hybrids with runtime guards. **Expected:** Each method has semantic limits. **Common mistake:** Assuming one example trace captures all control flow.

Q4. What is a graph break?

Answer: A point where compilation cannot safely represent or support the next program region, so execution returns to a fallback/eager runtime and may later re-enter compiled code. **Expected:** Lost optimization and boundary overhead. **Common mistake:** Claiming graph breaks are always correctness bugs.

Q5. What is shape inference?

Answer: Deriving output ranks/dimensions and constraints from input shapes and operation semantics, using static values, symbolic expressions, or runtime-dependent markers. **Expected:** Shapes drive legality, allocation, and specialization. **Common mistake:** Treating every unknown dimension as an exact compile-time constant.

Q6. What is graph partitioning?

Answer: Dividing the graph into regions assigned to different compilers, devices, libraries, or fallback execution. **Expected:** Include transfer/synchronization boundary costs. **Common mistake:** Maximizing node count on an accelerator without pricing boundaries.

Q7. How does a graph compiler choose between a library call and generated code?

Answer: It considers operator/shape support, layout, dtype, epilogues, expected performance, compilation cost, and numerical requirements. Mature dense operations often favor tuned libraries; unusual fused regions may favor code generation. **Expected:** Cost-based choice. **Common mistake:** Assuming generated code is inherently faster.

Q8. What is memory planning?

Answer: Assigning storage to graph values based on sizes, lifetimes, aliases, and execution order so safe buffers can be reused and peak memory reduced. **Expected:** Liveness. **Common mistake:** Reusing a buffer immediately after its last launch without considering asynchronous completion.

Q9. What is progressive lowering?

Answer: Converting from high-level graph/tensor representations through successively lower loop, GPU, and machine-oriented IRs while applying transformations at the level where needed information is explicit. **Expected:** Multiple IR levels. **Common mistake:** Jumping directly from graph nodes to SASS conceptually.

Q10. What is autotuning?

Answer: Generating or selecting among candidate algorithms/schedules, benchmarking them on representative inputs/hardware, and caching the best valid choice. **Expected:** Tuning overhead and cache validity. **Common mistake:** Benchmarking an unbounded search space during every request.

7. Deep-Dive Questions

1. How would you represent side effects in a graph?

Use explicit state values, effect/token edges, ordered regions, or conservatively placed barriers/graph breaks. Mutations need alias-aware dependencies; random operations need state sequencing; communication needs ordering and completion semantics. Pure data dependencies alone are insufficient.

2. How should symbolic shapes influence optimization?

Maintain constraints such as equality, divisibility, and bounds. Generate code valid for a symbolic range, guarded special cases for valuable patterns, and fallbacks for unsupported values. Avoid exact specialization on noisy dimensions unless measurements justify variants.

3. How can memory reuse be unsafe with GPU streams?

A host-side last use does not mean GPU work has completed. Reuse must respect stream order, cross-stream events, aliases, and library workspaces. A planner needs an execution-dependency model or synchronization before assigning overlapping storage.

4. Why are cost models difficult?

Runtime depends on shapes, layouts, caches, occupancy, launch overhead, library heuristics, neighboring operations, and hardware generation. Analytical estimates are cheap but imperfect; empirical tuning is accurate for measured cases but expensive and noisy. Practical compilers combine heuristics, models, and bounded tuning.

5. How do you validate graph-compiler correctness?

Compare against a trusted eager/reference path over diverse shapes, dtypes, layouts, alias patterns, control paths, and numerical extremes. Add metamorphic/property tests, gradient checks where relevant, differential fuzzing, and deterministic seeds. Validate guard failure and fallback, not only compiled happy paths.

8. Comparison Tables

Aspect		Eager execution	Graph compilation
Dispatch		Operator by operator	Optimized regions/plans
Global visibility		Low	High within captured graph
Startup/compile cost		Low	Potentially significant
Dynamic Python/control		Naturally handled	Must capture, guard, or break
Fusion/layout/memory planning		Limited/local	Cross-operation
Debugging		Direct operator boundaries	Generated/fused regions need correlation
Capture approach	Strength	Limitation	Good use
Example tracing	Simple, observes real ops	Misses unobserved data-dependent paths	Mostly static tensor programs
Source/bytecode analysis	Can retain control flow	Language complexity and unsupported features	Dynamic-language framework compilation
Explicit graph/export	Clear deployable contract	User/framework must express supported subset	Production inference/export

Capture approach	Strength	Limitation	Good use
Hybrid with guards	Balances dynamism and optimization	Recompilation/guard complexity	General framework JIT
Decision	Heuristic		
Generate kernel	Irregular/fused region where graph facts matter		
Call vendor library	Mature dense primitive with supported shape/layout/dtype		
Keep eager/fallback	Rare or unsupported operation, or compile cost exceeds benefit		
Move to CPU/other device	Only when compute and boundary transfer/sync cost justify it		

9. Common Mistakes

- Modeling the graph as pure data flow while ignoring mutation, randomness, and synchronization.
- Treating a sample trace as full program semantics.
- Fusing everything into a resource-heavy mega-kernel.
- Optimizing each node's layout without counting conversion costs across edges.
- Sending one unsupported node to CPU without pricing transfers and synchronization.
- Reusing buffers without stream-aware lifetime analysis.
- Over-specializing dynamic shapes and causing recompilation storms.
- Comparing compiled warm runs with eager cold runs or timing asynchronous launches incorrectly.
- Assuming graph compilation always means custom code generation; library selection is often best.

10. Edge Cases / Special Cases

- Data-dependent output shapes may require runtime allocation or multi-stage execution.
- Views and overlapping aliases can make apparently dead writes observable.
- In-place updates can block reordering and buffer reuse.
- Randomness, dropout, collective communication, host callbacks, and exceptions need explicit effect semantics.
- Quantized and mixed-precision operations carry scale, zero-point, accumulation, overflow, and rounding contracts.
- Training graphs include backward and optimizer state; saved activations create long lifetimes and checkpoint/recompute choices.
- Distributed graphs must coordinate computation, communication, streams, and failure behavior.
- Library algorithms may require temporary workspaces and have nondeterministic or numerical-mode constraints.

11. How to Explain in Interview

A graph compiler captures operations and dependencies into an IR so it can optimize across operator boundaries. It propagates shapes and constants, preserves effects, fuses profitable regions, chooses layouts and libraries, partitions across backends, plans memory, and progressively lowers to scheduled GPU kernels and native code. The hard parts are dynamic shapes, mutations/control flow, cost modeling, boundary costs, and proving guards and transformations correct.

12. Quick Revision Notes

- Graph compiler = capture + analyze + transform + partition/schedule + lower + execute.
- Graph edges may represent values, control, state, or effects.
- High-level passes: shape inference, folding, fusion, layouts, partitioning, memory planning.
- Low-level passes: tiling, thread mapping, vectorization, address spaces, code generation.
- Use libraries for strong primitives; generate code for valuable custom/fused regions.
- Dynamic behavior needs symbolic shapes, guards, variants, control-flow IR, or graph breaks.
- Interview traps: pure DAG assumptions and stream-unsafe memory reuse.

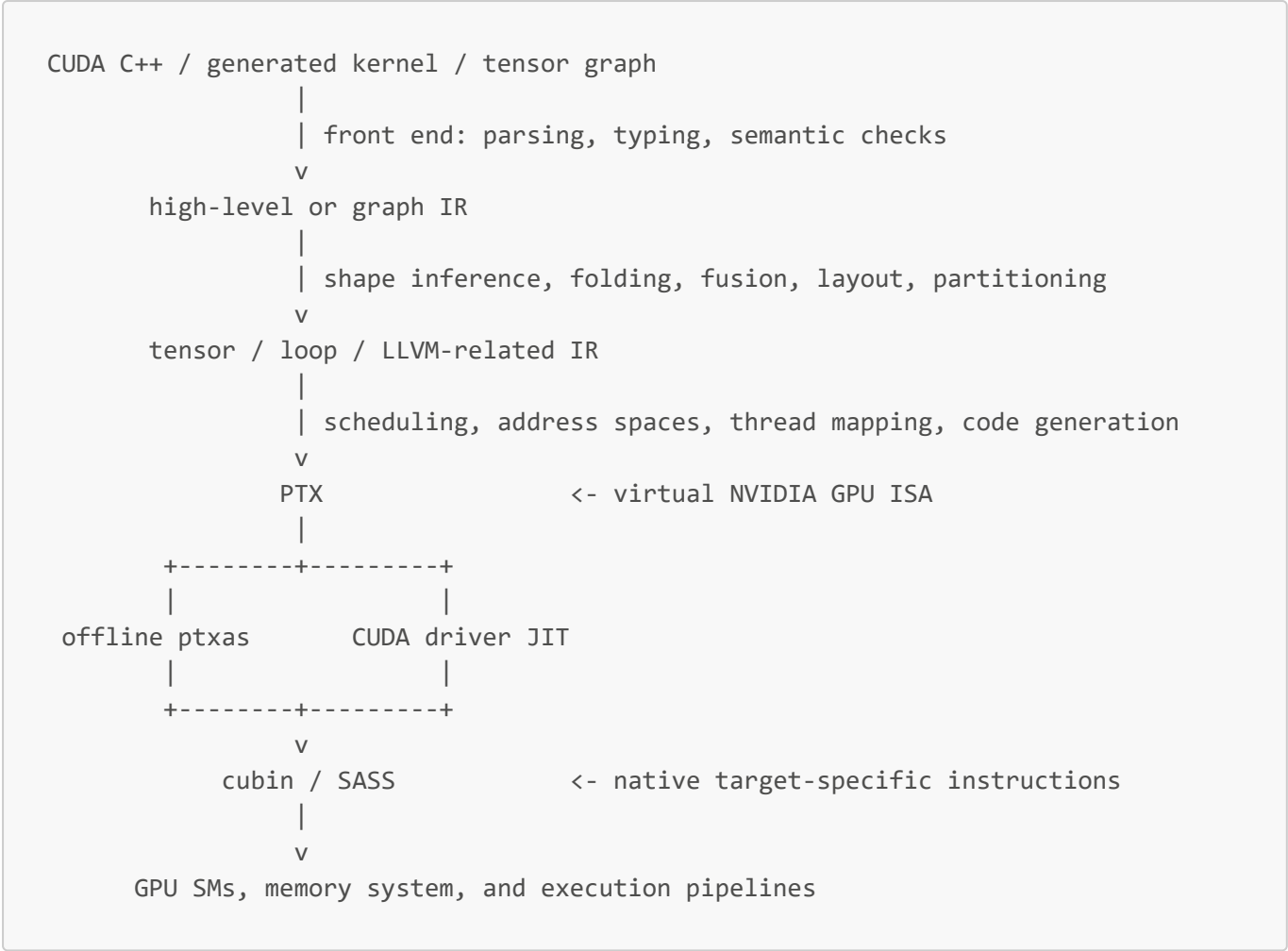
13. Practice Tasks

1. Draw a graph for `relu(x @ w + bias)` and mark possible library and fusion boundaries.
2. Implement constant folding and dead-node removal for a tiny toy DAG; preserve nodes marked side-effecting.
3. Given buffer lifetimes, perform a manual memory-reuse allocation, then add two asynchronous streams and revise it.
4. Design guards for a symbolic-batch, fixed-hidden-dimension linear layer.
5. Partition a graph with one unsupported CPU operator and calculate transfer cost versus keeping a larger region on GPU.
6. Compare three plans for a producer with multiple consumers: materialize, recompute, or fuse horizontally.
7. Build a differential test matrix covering shapes, strides, dtypes, edge sizes, NaN/Inf, mutation, and fallback.

14. Final Cheat Sheet

Item	Remember
Core definition	Whole-program/region compiler over operation and dependency graphs
Why it matters	Enables fusion, layouts, partitioning, memory planning, and specialization
Most asked	Capture; graph breaks; shapes; fusion; memory planning; lowering
Key comparison	Eager operator dispatch versus compiled optimized regions
Biggest trap	A data-flow DAG alone does not represent every effectful dynamic program
One-line answer	"A graph compiler uses whole-region visibility to turn tensor operations into a guarded, scheduled set of fused kernels and library calls."

End-to-End Stack Summary



Interview prompt	Strong first sentence
PTX vs SASS	PTX is a virtual ISA; SASS is target-native machine code.
What does <code>nvcc</code> do?	It drives separate host/device compilation and packages GPU images with host code.
Why JIT?	It trades cold compilation cost for runtime specialization and exact-target code.
Why fusion?	It removes launches and intermediate memory traffic when legality and resource costs permit.
Why graph compilers?	They expose cross-operator optimization, scheduling, library selection, and memory planning.
Where does LLVM fit?	It provides reusable SSA IR and lower-level optimization/code-generation infrastructure, often beneath richer tensor IRs.

Final Placement Checklist

Before an interview, be able to do all of the following without notes:

- 1. Draw the CUDA C++ → NVCC → PTX → SASS → GPU pipeline.
- 2. Explain virtual `compute_xy` versus real `sm_xy` targets.
- 3. Explain why PTX registers are not hardware registers.

4. Describe offline cubin generation and driver PTX JIT.
5. Explain host linking versus device linking.
6. Read a small SSA/CFG example and explain a phi node.
7. State two benefits and three possible costs of fusion.
8. Design guards and a cache key for a JIT specialization.
9. Explain graph capture, graph breaks, symbolic shapes, effects, and fallbacks.
10. Connect static inspection (IR/PTX/SASS) with dynamic evidence (profiler metrics and end-to-end timing).